Deep learning
Part I

Olivier Caelen

ULB UNIVERSITÉ LIBRE DE BRUXELLES
ECAM BRUSSELS ENGINEERING SCHOOL

Table of contents

What is deep learning

Artificial Intelligence

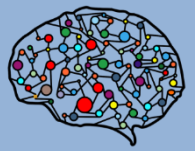
Any technique that allows the computer to imitate human behavior

Machine Learning

Ability to learn without being explicitly programmed

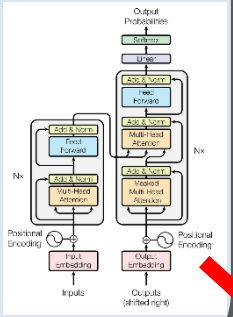
Deep learning

Extract patterns from data using artificial neural networks



Transformers

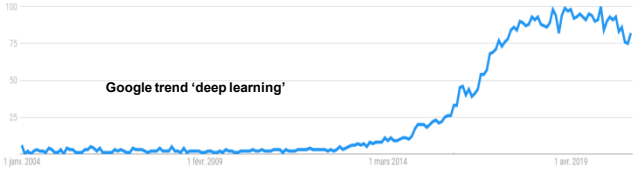
Models for sequence-to-sequence tasks using attention mechanisms



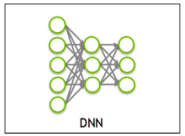
GPT is based on Transformers

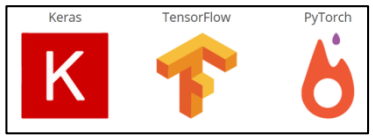
Deep learning Neural networks

- Deep learning systems are neural networks
  - Exist since the 70s (and even before).



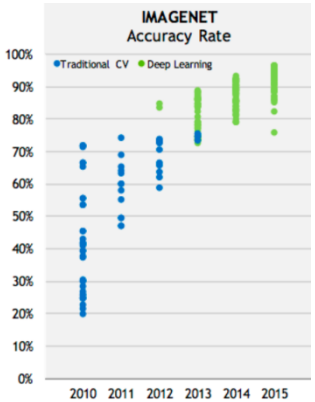
- What has changed:
  - more data
  - new hardware
  - new algorithms
  - new software

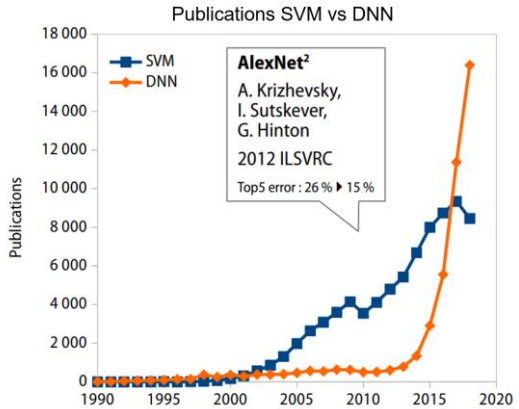


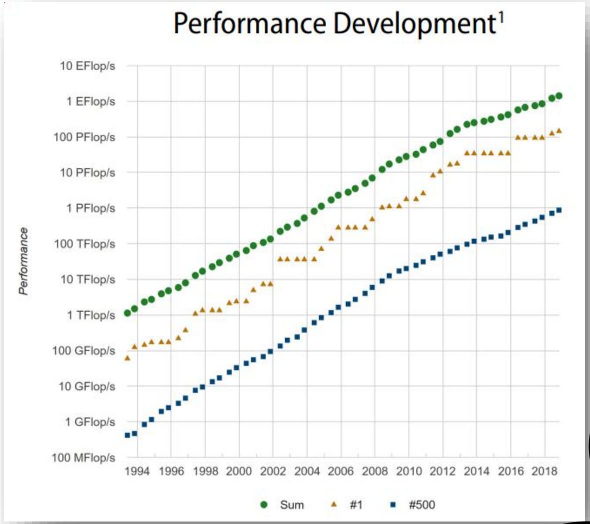


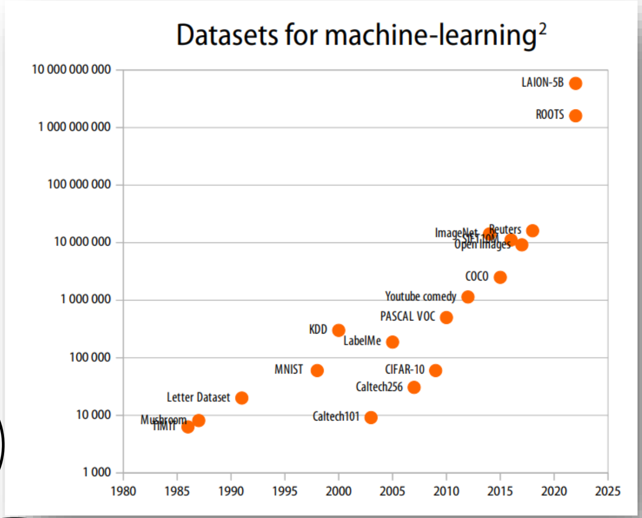
14,197,122 indexed images

IMAGENET









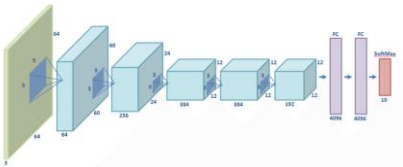
Source: TOP500 List - https://www.top500.org/

Source: Wikipedia

In 25 years

X 10^6 Flops

X 10^6 size

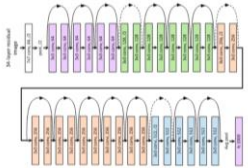


AlexNet

2012

- *Variational autoencoder* (**VAE**s) by Kingma & Welling

2013



- DL on images better than humans
- **ResNet**

2014

- *Generative adversarial network* (**GAN**s) by Goodfellow et al.

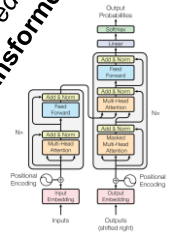
2015

- **AlphaGo** outperforms Go world champion

2016

- Paper: *Attention is All You Need*
- **Transformers**

2017



- Turing award for deep learning
- **Bert**
- **GPT-1**

2018

- **GPT-2**
- **StyleGANs**

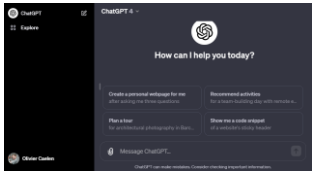
2019

- **GPT-3**

2020

- **AlphaFold2**
- **Dall-E 1**

2021



- **ChatGPT (~GPT 3.5)**
- **Stable Diffusion**
- **Dall-E 2**
- …

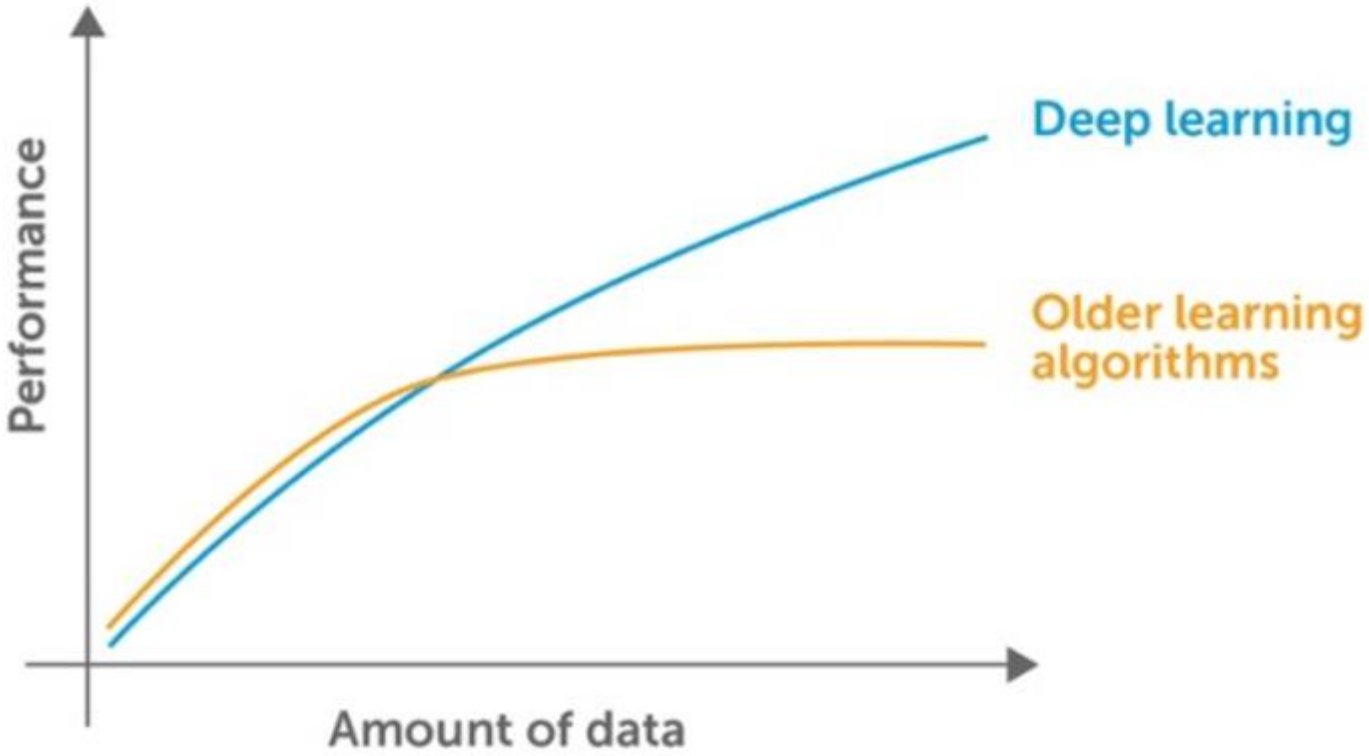
2022

- **GPT-4**
- **Llama2**
- **Dall-E 3**
- …

2023

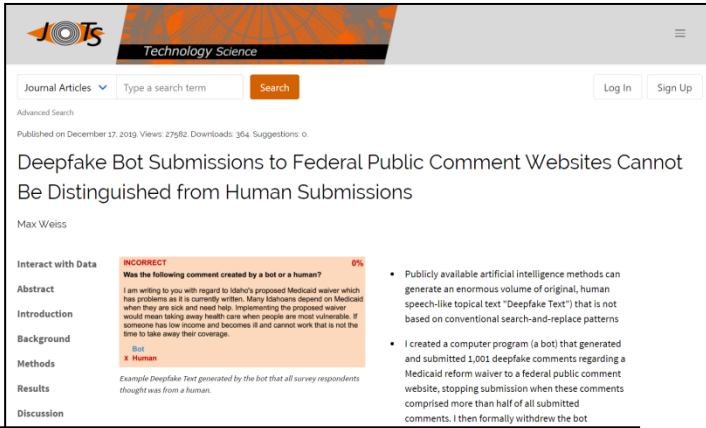
- **Generative AI**

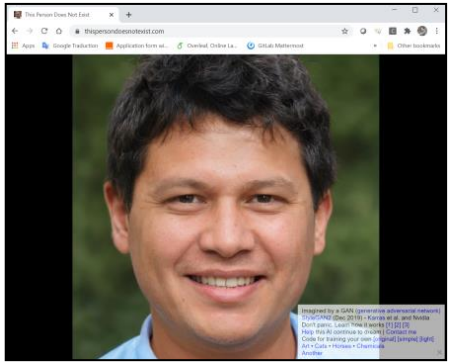
2024

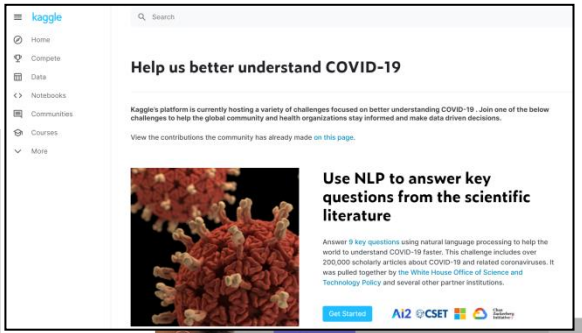


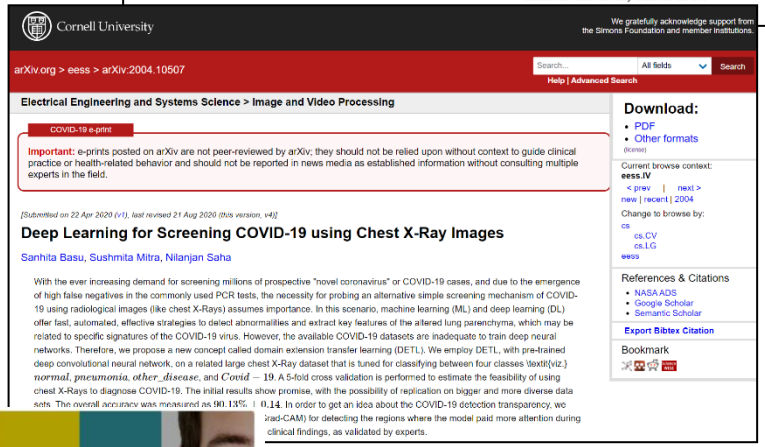
Credit: https://medium.datadriveninvestor.com/when-not-to-use-neural-networks-89fb50622429

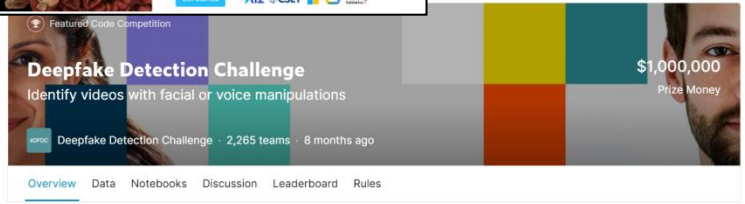
Deep learning everywhere











Hello Tensorflow

https://github.com/ocaelen/intro_deeplearning/blob/main/hello_tensorflow.ipynb

In [1]:
```
# Train a simple neural network for image classification
import tensorflow as tf
from tensorflow import keras
import numpy as np

print("Hello I'm tensorflow {}.".format(tf.__version__))

print('Loading data...')
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train= x_train.reshape(-1,28*28)
x_train = x_train/255.

print('Make the model...')
model = keras.models.Sequential([
  keras.layers.Dense(16, activation='relu'),
  keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

print('Training model...')
model.fit(x_train, y_train, epochs=3, batch_size=32)

print('Model trained successfully!')
```

Prepare the training set

Make the model

```
Hello I'm tensorflow 2.2.0.
Loading data...
Make the model
Training model...
Epoch 1/3
1875/1875 [==============================] - 14s 7ms/step - loss: 0.4853 - accuracy: 0.8591
Epoch 2/3
1875/1875 [==============================] - 13s 7ms/step - loss: 0.2778 - accuracy: 0.9199
Epoch 3/3
1875/1875 [==============================] - 13s 7ms/step - loss: 0.2398 - accuracy: 0.9301
Model trained successfully!
```

Hello PyTorch

```
import torch
import torch.nn as nn
import numpy as np
from sklearn import datasets
from pickletools import optimize
import matplotlib.pyplot as plt
```

```
## Prepare data
X_numpy, y_numpy = datasets.make_regression(n_samples=200, n_features=1, noise=10, random_state=42)
X = torch.from_numpy(X_numpy.astype(np.float32))
y = torch.from_numpy(y_numpy.astype(np.float32))
y = y.view(y.shape[0], 1)

n_samples, n_features = X.shape
```

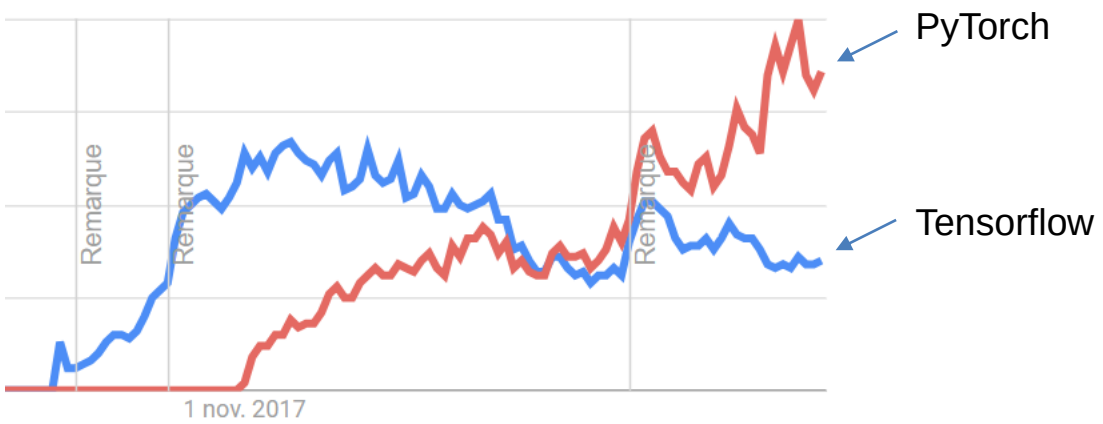
```
m = nn.Linear(n_features , 1) # input_size=1, output_size=1

loss_fct = nn.MSELoss()
optimizer = torch.optim.SGD(m.parameters(), lr=0.01)

for epoch in range(200):
    y_pred = m(X) # forward pass
    loss = loss_fct(y_pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if (epoch+1) % 40 == 0:
        print(f'epoch: {epoch+1}, loss:{loss.item():.4f}')
```

```
epoch: 40, loss:1757.2028
epoch: 80, loss:514.2151
epoch: 120, loss:204.7480
epoch: 160, loss:127.4618
epoch: 200, loss:108.1133
```

API docs

https://www.tensorflow.org/api_docs/python

https://pytorch.org/docs/stable/



Credit: Google Trends (Feb 2024)

Single-layer Neural Networks
(Linear regression, Perceptrons & Co…)

Question for you!

Which of these is **NOT** an example of supervised learning?

1. Predict whether a company will default on a loan.
2. Identify how many TV shows will a customer watch next month.
3. Build customer segments using Recency, Frequency and Monetary (RFM) values.
4. Predict which cars will need a mechanic given their technical characteristics.

Supervised training set

Input training set

$$\boldsymbol{X} = \begin{pmatrix} x_1^1 & \cdots & x_n^1 \\ \vdots & \ddots & \vdots \\ x_1^N & \cdots & x_n^N \end{pmatrix} = \begin{pmatrix} (\boldsymbol{x}^1)^T \\ \dots \\ (\boldsymbol{x}^N)^T \end{pmatrix} = (\boldsymbol{x}_1 \quad \dots \quad \boldsymbol{x}_n) \in \mathbb{R}^{N \times n}$$

where

- N is the number of training observations
- n is the number of input features

Output corresponding values (supervised learning)

$$\boldsymbol{t} = \begin{pmatrix} t^1 \\ \dots \\ t^N \end{pmatrix}$$

where

- $t^i \in \mathbb{R}, \ \forall i \in \{1 \dots N\}$ → regression
- $t^i \in \{C_1 \dots C_K\}, \ \forall i \in \{1 \dots N\}$ → Classification

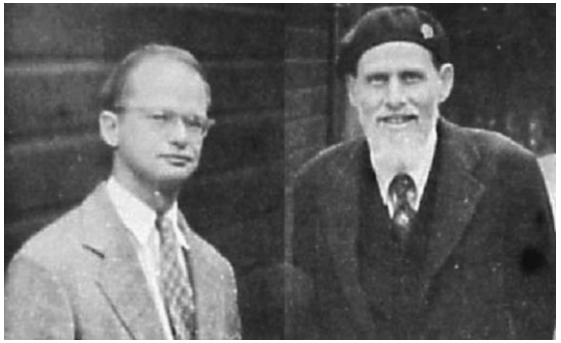
Note that the output dimensionality could be > 1. In this case, M stands for this dimension.

- Sometimes, the training set will be rewritten as a set of tuples of input vectors and output values

$$D_N = \{(\boldsymbol{x}^i, t^i)\}_{1 \le i \le N} \quad \text{where} \quad \boldsymbol{x}^i \in \mathbb{R}^n$$

- Bold values :
  - uppercase (e.g., $\boldsymbol{X}$) → matrix
  - lowercase (e.g., $\boldsymbol{x}^i$) → vector

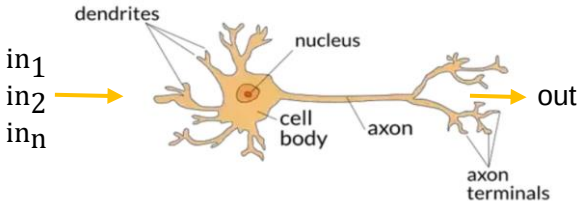
HISTORY: A first ARTIFICIAL NEURAL NETWORK

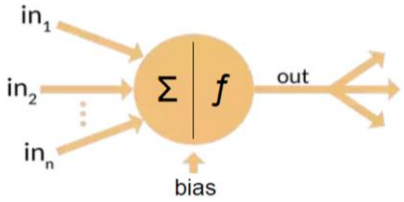


McCulloch & Pitts 1943

Frank Rosenblatt

1958





A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH and WALTER H. PITTS

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

Bulletin of Mathematical Biophysics; 1943

This paper provided a way to describe brain functions in abstract terms and showed that simple elements connected in a neural network can have immense computational power.

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN[1]

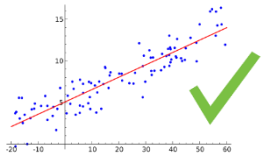
F. ROSENBLATT

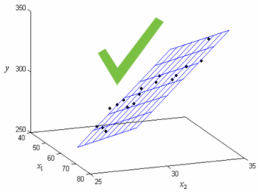
Cornell Aeronautical Laboratory

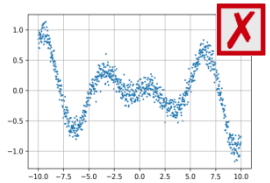
Psychological Review; 1958

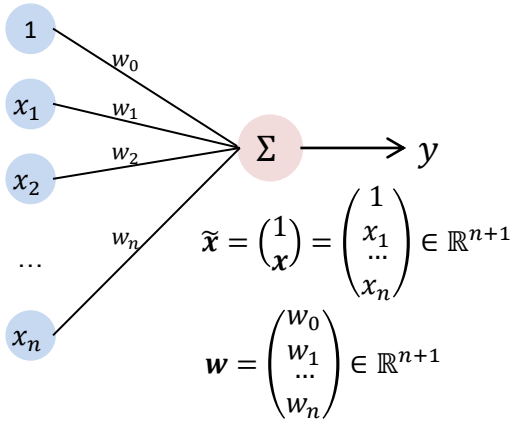
Linear regression

- Probably the most elementary way to perform regression.
- It is a linear model (cannot capture non linear relations)









- Let $J(\boldsymbol{w}, D_N)$ be the loss function that we can define in regression as follow

$$J(\boldsymbol{w}, D_N) = \frac{1}{N}\sum_{i=1}^{N}\left(h(\boldsymbol{x}^i) - t^i\right)^2 = \frac{1}{N}\sum_{i=1}^{N}\left(\boldsymbol{w}^T\tilde{\boldsymbol{x}}^i - t^i\right)^2 = \frac{1}{N}\left\|\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right\|^2$$

where $\tilde{\boldsymbol{X}} = \begin{pmatrix} 1 & \\ \vdots & \boldsymbol{X} \\ 1 & \end{pmatrix} \in \mathbb{R}^{N\times(n+1)}$

$$h(\boldsymbol{x}) = \boldsymbol{w}^T\tilde{\boldsymbol{x}} = w_0 + \sum_{j=1}^{n} x_j w_j$$

where "h"stands for *hypothesis*

- This loss function is called the *mean squared error* (MSE)
- The most used in regression problems (why?)
- Note that, the fraction $\frac{1}{N}$ is often omitted *or replaced by* $\frac{1}{2}$

Learning: Given a training set D_N, search the optimal parameters $\boldsymbol{w}$ minimizing $J(\boldsymbol{w}, D_N)$.

Learning in linear regression context

Learning: Given a training set D_N, search the optimal parameters $\boldsymbol{w}$ minimizing $J(\boldsymbol{w}, D_N)$.

- Learning is an optimization problem.
  - Therefore, many optimization algorithms can be used to find the best $\boldsymbol{w}$.

- In the next slide, we will search analytically the parameters for which the gradient of $J(\boldsymbol{w}, D_N)$ is zero
  - i.e., $\left(\frac{\partial J}{\partial \boldsymbol{w}}\right) = 0$
  - Note, a direct analytical solution is possible due to the fact when a linear model is used (i.e., $h(\boldsymbol{x}) = \boldsymbol{w}^T \tilde{\boldsymbol{x}}$) then the second derivatives $\left(\frac{\partial^2 J}{\partial (w_1)^2}\right)$ and $\left(\frac{\partial^2 J}{\partial (w_2)^2}\right)$ are always positive (not showed in this course)
    → therefore, $J(\boldsymbol{w}, D_N)$ is a convex function → only one minimum

Optimizing the criterion by pseudo-inverse

- Error criterion $J = \frac{1}{2}\left\|\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right\|^2$ (Note, for simplicity I replaced $^1/_N$ by $^1/_2$)
- Gradient of error (function to minimize) with respect to weights (free parameters)

$$\nabla_{\boldsymbol{w}} J = \left(\frac{\partial J}{\partial \boldsymbol{w}}\right) = \left(\frac{\partial J}{\partial w_0}, \frac{\partial J}{\partial w_1}, \ldots, \frac{\partial J}{\partial w_n}\right)^T \in \mathbb{R}^{n+1}$$

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j}\left(\frac{1}{2}\left\|\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right\|^2\right) = \frac{2}{2}\left(\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right)^T \underbrace{\left(\frac{\partial}{\partial w_j}\tilde{\boldsymbol{X}}\boldsymbol{w}\right)} = \left(\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right)^T \boldsymbol{x}_j$$

$$\nabla_{\boldsymbol{w}} J = \left(\left(\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right)^T \tilde{\boldsymbol{X}}\right)^T = \tilde{\boldsymbol{X}}^T\left(\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right)$$

$$\frac{\partial}{\partial w_j}\tilde{\boldsymbol{X}}\boldsymbol{w} = \frac{\partial}{\partial w_j}\begin{pmatrix} 1 x_1^1 & \cdots & x_n^1 \\ \vdots\ \vdots & \ddots & \vdots \\ 1 x_1^N & \cdots & x_n^N \end{pmatrix}\begin{pmatrix} w_0 \\ w_1 \\ \cdots \\ w_n \end{pmatrix} = \frac{\partial}{\partial w_j}\begin{pmatrix} w_0 + x_1^1 w_1 + \ \ldots + x_j^1 w_j + \cdots + x_n^1 w_n \\ \vdots \\ w_0 + x_1^N w_1 + \ \ldots + x_j^N w_j + \cdots + x_n^N w_n \end{pmatrix} = \begin{pmatrix} x_j^1 \\ x_j^2 \\ \cdots \\ x_j^N \end{pmatrix}$$

- Minimum of error

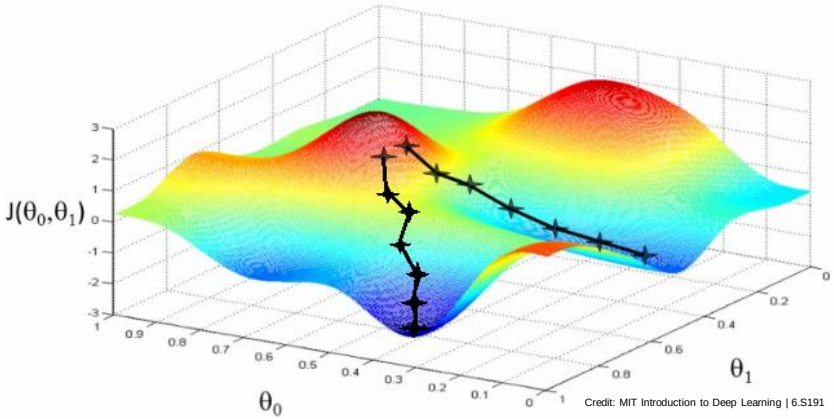
$$\left(\frac{\partial J}{\partial \boldsymbol{w}}\right) = \boldsymbol{0}$$
$$\Leftrightarrow \tilde{\boldsymbol{X}}^T\left(\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right) = \boldsymbol{0}$$
$$\Leftrightarrow \tilde{\boldsymbol{X}}\boldsymbol{w} = \boldsymbol{t}$$
$$\Leftrightarrow \tilde{\boldsymbol{X}}^T\tilde{\boldsymbol{X}}\boldsymbol{w} = \tilde{\boldsymbol{X}}^T\boldsymbol{t}$$
$$\Leftrightarrow \left(\tilde{\boldsymbol{X}}^T\tilde{\boldsymbol{X}}\right)^{-1}\tilde{\boldsymbol{X}}^T\tilde{\boldsymbol{X}}\boldsymbol{w} = \left(\tilde{\boldsymbol{X}}^T\tilde{\boldsymbol{X}}\right)^{-1}\tilde{\boldsymbol{X}}^T\boldsymbol{t}$$
$$\Longrightarrow \hat{\boldsymbol{w}} = \left(\tilde{\boldsymbol{X}}^T\tilde{\boldsymbol{X}}\right)^{-1}\tilde{\boldsymbol{X}}^T\boldsymbol{t}$$

Pseudo-inverse requires:
- All input-output pairs from D_N
- matrix inversion (often ill-configured) (why?)

Necessity for iterative methods without matrix inversion
→ Gradient descent !

Training: Batch gradient descent (BGD)

General form: $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \alpha \frac{\partial J}{\partial \boldsymbol{w}}|_{\boldsymbol{w}(t)}$



- The value of the gradient $\nabla_{\boldsymbol{w}} J$ at $\boldsymbol{w}(t)$ gives the *local* direction of the steepest *increasing* slope.
- At each step t, we move in the *opposite* direction of the gradient.
- The parameter α (*learning rate*) is used to adapt the step sizes when moving.

When error criterion J equals $\frac{1}{2}\left\|\tilde{\boldsymbol{X}}\boldsymbol{w} - \boldsymbol{t}\right\|^2$

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \alpha\, \tilde{\boldsymbol{X}}^T\left(\boldsymbol{t} - \tilde{\boldsymbol{X}}\boldsymbol{w}(t)\right)$$

Pseudo-inverse and gradient descent:
Same error criterion → gradient descent tends to the same solution.

Pros & cons
- does not require matrix inversion
- still needs all input-output pairs from D_N

Training: one sample stochastic gradient descent (1-sample SGD)

If data are *stationery*, **then** minimizing J is equivalent to successively minimizing each J^k

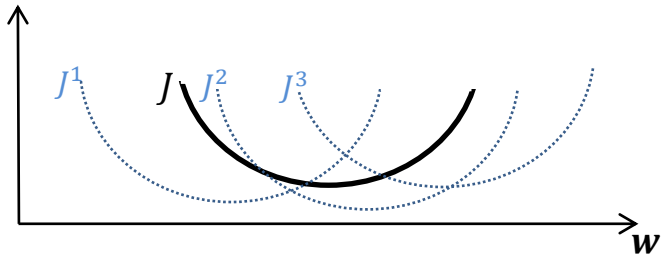
$$J(\boldsymbol{w}, D_N) = \frac{1}{2}\sum_{i=1}^{N}\left(h(\boldsymbol{x}^i) - t^i\right)^2 = \frac{1}{2}\sum_{i=1}^{N} J^i$$

The formula for the update of linear regression models in the (stochastic) gradient descent becomes:

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \alpha\left(t^k - \boldsymbol{w}(t)^T\tilde{\boldsymbol{x}}^k\right)\tilde{\boldsymbol{x}}^k$$

Only one observation is used at each step k.

Difference between i, k :
- i and k are indices on the observations (1…N)
- i is the index in the database of observations, while k identifies the order of presentation, which my differ. The difference between i and k is not crucial here.



1-sample stochastic gradient descent

Initializes $\boldsymbol{w}(0)$

Repeat

  Select $(\boldsymbol{x}^k, t^k)$ from D_N

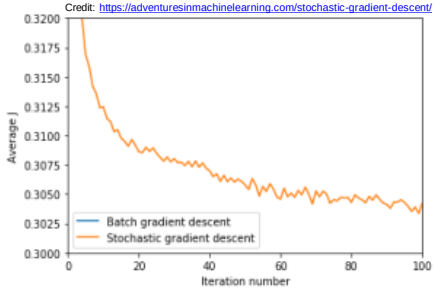
  - Forward pass
    Compute output
    $h(\boldsymbol{x}^k) = \boldsymbol{w}(t)^T\tilde{\boldsymbol{x}}^k$

  - Backward pass
    Compute gradient
    $\nabla_{\boldsymbol{w}}J^k = \left(h(\boldsymbol{x}^k) - t^k\right)\tilde{\boldsymbol{x}}^k$

  - Update parameters
    $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \alpha\,\nabla_{\boldsymbol{w}}J^k$

Training: Mini-batch gradient descent

Credit: https://adventuresinmachinelearning.com/stochastic-gradient-descent/



Note about 1-sample SGD: As we are only considering *one* example at a time, the cost will fluctuate based on the training examples and it will not necessarily decrease. But in the long run, you will see the cost decrease with the fluctuations.

Mini-batch gradient descent is a trade-off between 1-sample SGD and the BGD.

In mini-batch gradient descent, the cost function (and therefore gradient) is averaged over a small number of samples (e.g., 10 - 500).

This is opposed to the SGD batch size of *1* sample, and the BGD size of *all* the training samples.

The default **batch_size** in Keras is 32

Note: in many sources, the terms *mini-batch* and *stochastic gradient descent* are synonymous.

Question for you!

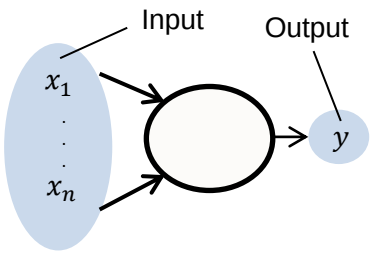
Neural networks are rarely optimized using Batch Gradient Descent (BGD), and instead we use mini-batches from the dataset to train on (often implemented as Stochastic Gradient Descent, or SGD). What are the advantages of this?

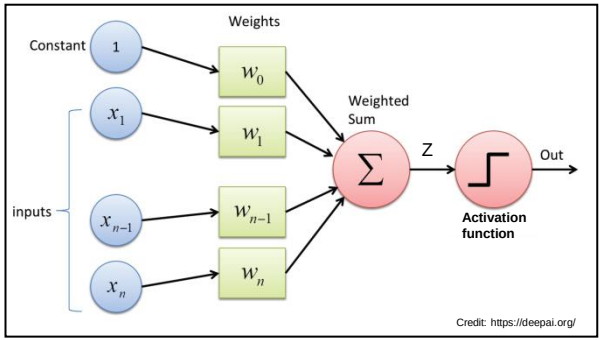
Select all that apply.

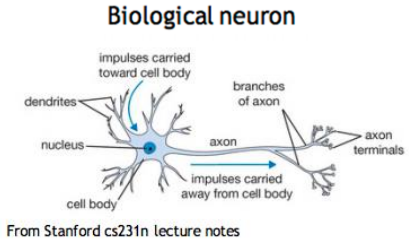
A. BGD is difficult to use on large datasets because we have to store the entire dataset in memory before performing a parameter update.
B. SGD is able to jump out of local minima that would otherwise trap BGD.
C. SGD gives a better approximation to the true gradient.

Perceptron

- Probably the most elementary way to perform binary classification
- The basic computing unit in neural networks : 'neuron'
  - Example: Perceptron
- A **perceptron** ≈ a neural net with only **one neuron**.







Vocabulary:

- $f()$ is the **activation function** of the neuron.
- $(w_0, w_1, \dots, w_n)^T$ are the **synaptic weights**.
- w_0 is also called the **threshold** (Bias value).

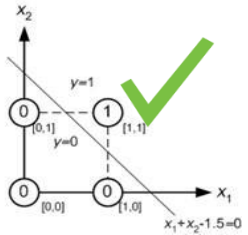
During the **learning** process, we search the optimal values of the **synaptic weights.**

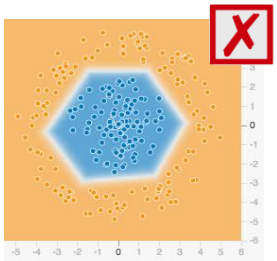
$$z = \sum_{j=1}^{n} w_j x_j + w_0 = \boldsymbol{w}^T \tilde{\boldsymbol{x}}$$

$$out = f(z) = \begin{cases} 1 \; if \; z \geq 0 \\ 0 \; if \; z < 0 \end{cases}$$

It is a linear separator

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1





Perceptron - learning

- Class label $t^k \in \{0,1\}$ and perceptron output $h(\boldsymbol{x}^k) \in \{0,1\}$ where $f(z) = \begin{cases} 1 \; if \; z \geq 0 \\ 0 \; if \; z < 0 \end{cases}$ is the activation function.
- Let us define a new variable $d = \begin{cases} 1 \; if \; t = 1 \\ -1 \; if \; t = 0 \end{cases}$
- We know that <u>in case of *correct* classification</u>:
  - if $t^k = 1$ then $\boldsymbol{w}^T \tilde{\boldsymbol{x}}^k \geq 0$
  - if $t^k = 0$ then $\boldsymbol{w}^T \tilde{\boldsymbol{x}}^k < 0$

  $\Longrightarrow \quad \left(\boldsymbol{w}^T \tilde{\boldsymbol{x}}^k\right) d^k \geq 0$
- An ideal criterion could be $J = \sum_{\left(\boldsymbol{w}^T \tilde{\boldsymbol{x}}^i\right) d^i < 0} 1$ but this error criterion is not continuous (gradient can not be computed).
- Perceptron criterion (is continuous): $J = \sum_{\left(\boldsymbol{w}^T \tilde{\boldsymbol{x}}^i\right) d^i < 0} -1 \times \left(\boldsymbol{w}^T \tilde{\boldsymbol{x}}^i\right) d^i = \sum_{i=1}^{N} J^i$ where $J^i = \begin{cases} -1 \times \left(\boldsymbol{w}^T \tilde{\boldsymbol{x}}^i\right) d^i & \text{if misclassification} \\ 0 & \text{if } \textit{correct} \text{ classification} \end{cases}$
- Stochastic gradient descent on perceptron learning rule:
  - <u>If $\tilde{\boldsymbol{x}}^k$ is misclassified</u> (and only in this case):

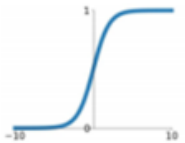
$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \alpha \, \tilde{\boldsymbol{x}}^k d^k$$

Activation function

- The activation function takes as input the weighed sum of the input (i.e., $z = \boldsymbol{w}^T \tilde{\boldsymbol{x}}$).
- The step function $f(\mathrm{z}) = \begin{cases} 1 \; if \; z \geq 0 \\ 0 \; if \; z < 0 \end{cases}$ was originally used as activation function in the perceptron.
- Note that
  - if $f(\mathrm{z}) = z$ is used as activation function then the *perceptron* becomes *linear regression*
  - if sigmoid $\sigma(z)$ is used than the *perceptron* becomes *logistic regression*

Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



Linear regression with TensorFlow (Keras)

https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_lin_reg.ipynb

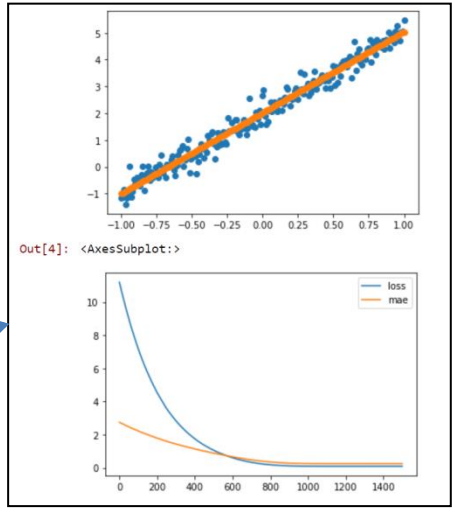
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers.experimental import preprocessing
```

In [2]:
```
x = np.linspace(-1,1, 200)
y = 3*x + 2 + np.random.normal(size = len(x),scale=0.3)
x = x.reshape((-1,1))
y = y.reshape((-1,1))
```

In [3]:
```
model = Sequential([
    Dense(1,input_shape=(1,)) # Default activation: 'linear'
])
model.summary()
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
hist = model.fit(x, y, epochs=1500, verbose=0)
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 1)                 2
=================================================================
Total params: 2
Trainable params: 2
Non-trainable params: 0
_________________________________________________________________
```

In [4]:
```
y_hat=model.predict(x)

plt.scatter(x,y)
plt.scatter(x,y_hat)
plt.show()

pd.DataFrame(hist.history).plot()
```



Linear regression with PyTorch

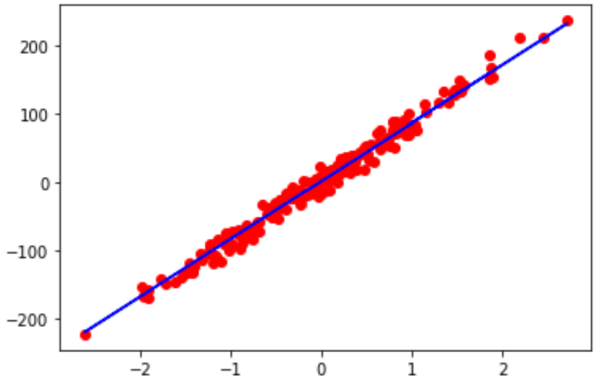
```
import torch
import torch.nn as nn
import numpy as np
from sklearn import datasets
from pickletools import optimize
import matplotlib.pyplot as plt
```

```
## Prepare data
X_numpy, y_numpy = datasets.make_regression(n_samples=200, n_features=1, noise=10, random_state=42)
X = torch.from_numpy(X_numpy.astype(np.float32))
y = torch.from_numpy(y_numpy.astype(np.float32))
y = y.view(y.shape[0], 1)

n_samples, n_features = X.shape
```

```
m = nn.Linear(n_features , 1) # input_size=1, output_size=1

loss_fct = nn.MSELoss()
optimizer = torch.optim.SGD(m.parameters(), lr=0.01)

for epoch in range(200):
    y_pred = m(X) # forward pass
    loss = loss_fct(y_pred, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if (epoch+1) % 40 == 0:
        print(f'epoch: {epoch+1}, loss:{loss.item():.4f}')
```

```
epoch: 40, loss:1757.2028
epoch: 80, loss:514.2151
epoch: 120, loss:204.7480
epoch: 160, loss:127.4618
epoch: 200, loss:108.1133
```

```
predicted = m(X).detach().numpy()
plt.plot(X_numpy, y_numpy, 'ro')
plt.plot(X_numpy, predicted, 'b')
plt.show()
```



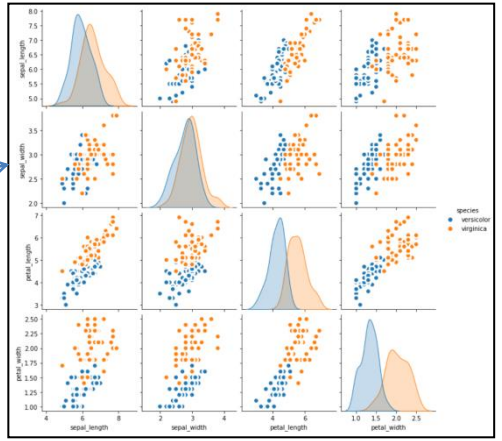
https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_logistic_reg.ipynb

Logistic regression with TensorFlow (Keras)

In [1]:
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import pandas as pd
```

In [2]:
```
df = sns.load_dataset('iris')

## Delete a class to have a binary classification problem
df = df[df['species'] != 'setosa']
```

In [3]:
```
sns.pairplot(df, kind="scatter", hue="species", plot_kws=dict(s=80, edgecolor="white", linewidth=2.5))
plt.show()
```



In [4]:
```
X = df.iloc[:,0:4].values
X = StandardScaler().fit_transform(X)

y = (df.iloc[:,4].values == 'versicolor').astype(int)

print(X[:5,:])
print(y[:5])
```

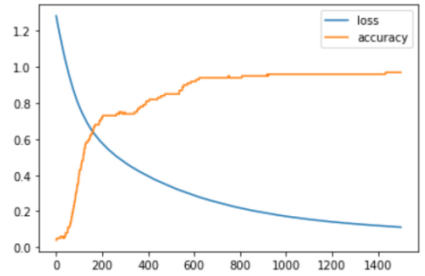
```
[[ 1.11900931  0.99068792 -0.25077906 -0.65303909]
 [ 0.20924564  0.99068792 -0.49425387 -0.41643072]
 [ 0.96738203  0.68864892 -0.00730424 -0.41643072]
 [-1.15539985 -1.72766308 -1.10294091 -0.88964745]
 [ 0.36087292 -0.21746808 -0.37251647 -0.41643072]]
[1 1 1 1 1]
```

In [5]:
```
model = Sequential([
    Dense(1,input_shape=(4,), activation='sigmoid') # Default activation: 'linear'
])
model.summary()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
hist = model.fit(X, y, epochs=1500, verbose=0)
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 1)                 5
=================================================================
Total params: 5
Trainable params: 5
Non-trainable params: 0
_________________________________________________________________
```

In [6]:
```
pd.DataFrame(hist.history).plot()
test_loss, test_accuracy=model.evaluate(X,y, verbose=0)
print(f'Test loss: {test_loss}')
print(f'Test accuracy: {test_accuracy}')
```

```
Test loss: 0.11126507073640823
Test accuracy: 0.9700000286102295
```



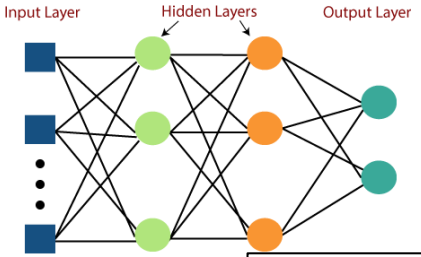
Multilayer perceptron

Multilayer perceptron (MLP)

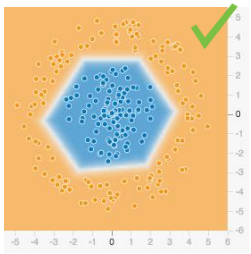
(also called *Feed-forward neural networks* or *Plain vanilla*)

- Combine input information in a complex & flexible neural net model

- Network Structure
  - Multiple layers
    - Input layer
    - Hidden layer(s)
    - Output layer



Warning: this representation don't show the bias values!

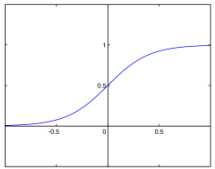


Question for you!

Why is it needed to add non linear activation function?
What if all activation functions are linear?

- Multilayer perceptron → fully connected layers

- Activation functions f in the hidden layers introduces nonlinearity.
  - Historically, the most common activation functions in the hidden layers was sigmoid.



$$f(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Old school! For hidden layers in MLP use ReLu

$\sigma(z)$ transform any input $z \in \mathbb{R}$ into a number in $[0, 1]$.

Activation function

Some other popular examples of activation functions

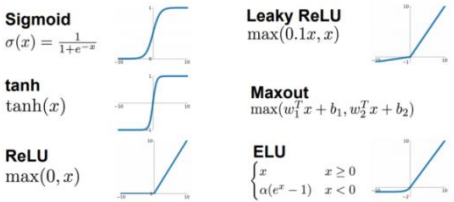


Image taken from Stanford's CS231n class

But… How to choose the activation function that we will use in our deep learning model ??

There are no general rules but some *common* rules depending on the type of layers (input layer, hidden layer(s) and output layer)

→ For the input layer, it's simple: As input layers takes raw input from the domain, there are no activation functions in the input layer.

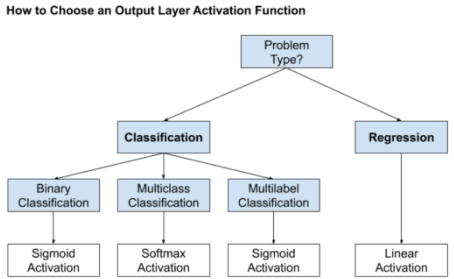
Activation for Hidden Layers

There might be three activation functions that you might want to consider using in hidden layers; they are:

- Logistic (Sigmoid)
  - Traditionally, the sigmoid activation function was the default activation function in the 1990s.
  - Still used in some advance cases (e.g., recurrent neural network)
- Hyperbolic Tangent (Tanh)
  - Still used in some advance cases (e.g., recurrent neural network)
- Rectified Linear Activation (ReLU)
  - *"In modern neural networks, the default recommendation is to use the rectified linear unit or ReLU (…)"* I. Goodfellow et al. **Deep Learning,** 2016

A MLP neural network will almost always have the same activation function in all hidden layers.

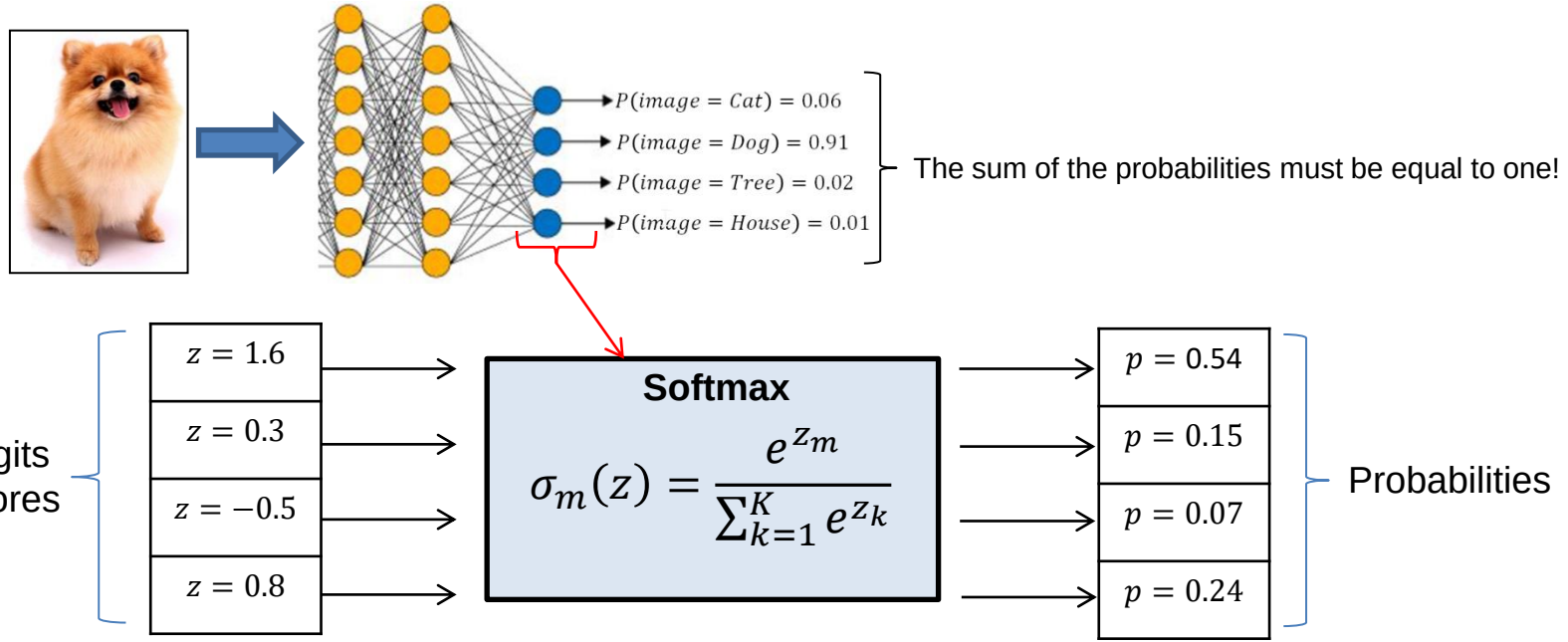
Activation for Output Layers



Credit: machinelearningmastery.com/

Activation function – Softmax

- ***Softmax*** is a kind of generalization of the logistic function for the *multiclass* classification problem.
- In a *multiclass* classification problem:
  - the target variable can take many distinct discrete values : $t^i \in \{C_1 \dots C_K\}$ where K is the number of class.
  - The output of the model $h(\boldsymbol{x})$ is a vector of values in [0,1] which sum to 1 (i.e. probabilities)



Homework [optional!]

Youtube

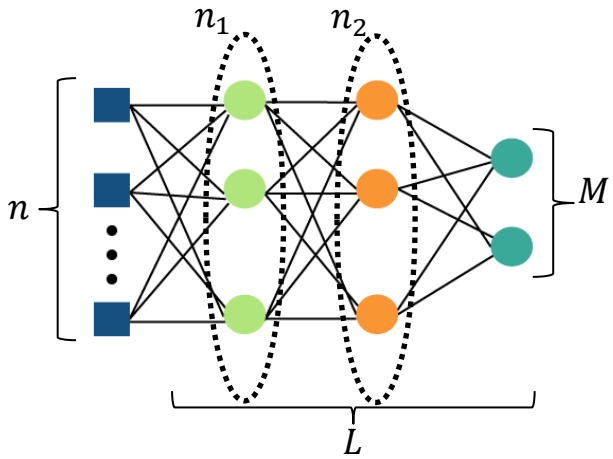
- **3Blue1Brown** But what is a Neural Network? | Deep learning, chapter 1
  https://www.youtube.com/watch?v=aircAruvnKk

- **3Blue1Brown** Gradient descent, how neural networks learn | Deep learning, chapter 2
  https://www.youtube.com/watch?v=IHZwWFHWa-w

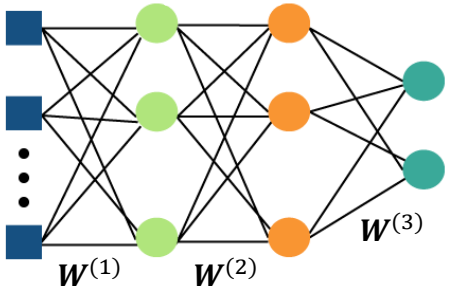
Please send a summary of the two videos:

- What did you learn as new concepts that are not in the slides?
- One A4 page max.
- Deadline: in 3 weeks
- One extra point (+1/20) in addition to the oral assessment

Multilayer perceptron – notation



- n : number of input features.
- $n_1, n_2, \ldots, n_l, \ldots$: number of nodes in layers
- M : number of output values
- L : number of layers.



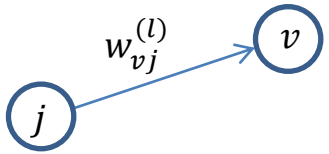
$\boldsymbol{W}^{(l)} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$: synaptic weights between layers $(l-1)$ and l.

Example, if $n = 1$ and $n_1 = 3$ then

$$\boldsymbol{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \\ w_{31}^{(1)} & w_{32}^{(1)} \end{pmatrix} \in \mathbb{R}^{3\times 2}$$

Multilayer perceptron – notation

Let $w_{vj}^{(l)}$ be the synaptic weight between neuron j and neuron v in layer l.



with: $\boldsymbol{W}^{(\boldsymbol{l})} = \begin{pmatrix} w_{1,1}^{(l)} & \cdots & w_{1,(n_{l-1}+1)}^{(l)} \\ \vdots & w_{v,j}^{(l)} & \vdots \\ w_{n_l,1}^{(l)} & \cdots & w_{n_l,(n_{l-1}+1)}^{(l)} \end{pmatrix} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$

Hyperparameters

The “structure” of a multilayer perceptron is characterized by many hyperparameter

- number of hidden layers
- number of nodes (neurons) per hidden layer
- activation functions in each layer

These hyperparameters are defined (chosen) by the data scientist.

  - Note that hyperparameter identification by ‘cross-validation like’ methods is difficult with DL

Others are constrained by the modeling problem e.g.,

- number of input features
- number of output values
- activation function in the last layer (i.e., regression or classification)

Expressiveness : One-hidden layer vs many hidden layers (A.k.a. deep learning)

- A one Hidden Layer Neural Network Architecture with sigmoid/hyperbolic units and with enough neurons in its hidden layer (even ∞ neurones if necessary) can approximate arbitrarily closely any function
  -> Universal approximation

- However, as we increase the number of layers, the number of units needed can decrease (even exponentially) with the number of layers.

Questions for you!

Draw the following neural network:

- $L = 3$
- $n = 1$
- $n_1 = 2$
- $n_2 = 3$
- $M = 1$

How many synaptic weights (tips: don't forget the bias values ;-))?

A. *12* synaptic weights
B. *17* synaptic weights
C. *49* synaptic weights
D. *15* synaptic weights

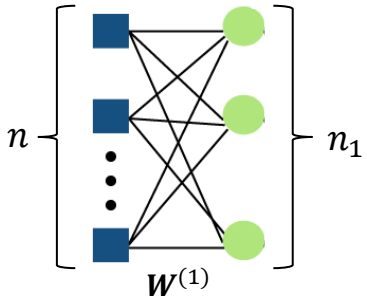
How many trainable parameters does a feedforward network have with input shape (64,), three hidden layers with 16 units each and a final linear layer with 8 units?

- 1720
- 968
- 7416
- 142

Why do we use non-linear activation functions (such as *tanh* or *relu*) in neural networks?

A. To allow the usage of higher learning rates, thus speeding up the convergence during the optimization.
B. To induce sparse connectivity in the network weights.
C. Without non-linear activation functions, the network would only be able to model linear functions of the data.
D. So that the model activations are equivariant with respect to the input features

Multilayer perceptron – formula

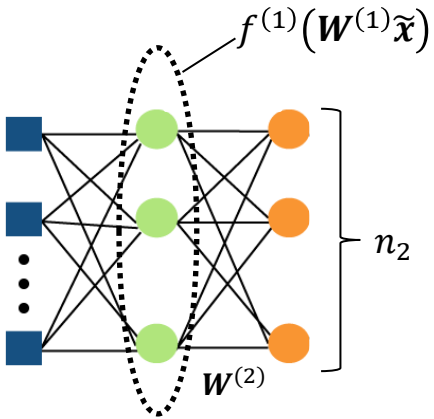


$$\boldsymbol{W}^{(1)} \in \mathbb{R}^{n_1 \times (n+1)} \quad \rightarrow \quad \boldsymbol{W}^{(1)}\,\tilde{\boldsymbol{x}} \in \mathbb{R}^{n_1 \times 1} \quad \rightarrow \quad f^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right) = \boldsymbol{a}_1 = \begin{pmatrix} a_1^{(1)} \\ \vdots \\ a_{n_1}^{(1)} \end{pmatrix} \in \mathbb{R}^{n_1 \times 1}$$

$$\tilde{\boldsymbol{x}} \in \mathbb{R}^{(n+1)\times 1}$$

$a_j^{(l)}$ is the output of node j in layer l

Multilayer perceptron – formula

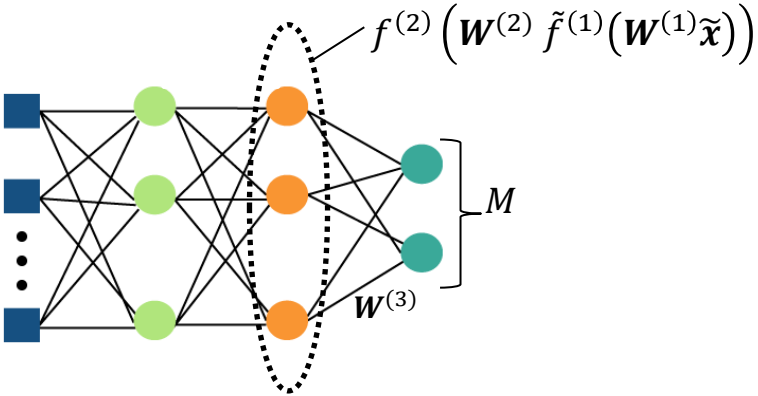


$\boldsymbol{W}^{(2)} \in \mathbb{R}^{n_2 \times (n_1+1)}$ → $\boldsymbol{W}^{(2)}\, \tilde{f}^{(1)}(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}) \in \mathbb{R}^{n_2 \times 1}$ → $f^{(2)}\left(\boldsymbol{W}^{(2)}\, \tilde{f}^{(1)}(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}})\right) = \begin{pmatrix} a_1^{(2)} \\ \vdots \\ a_{n_2}^{(2)} \end{pmatrix} \in \mathbb{R}^{n_2 \times 1}$

$f^{(1)}(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}) \in \mathbb{R}^{n_1 \times 1}$

where $\tilde{f}^{(1)}(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}) = \begin{pmatrix} 1 \\ f^{(1)}(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}) \end{pmatrix} \in \mathbb{R}^{(n_1+1) \times 1}$

Multilayer perceptron – formula



$\boldsymbol{W}^{(3)} \in \mathbb{R}^{M\times(n_2+1)}$

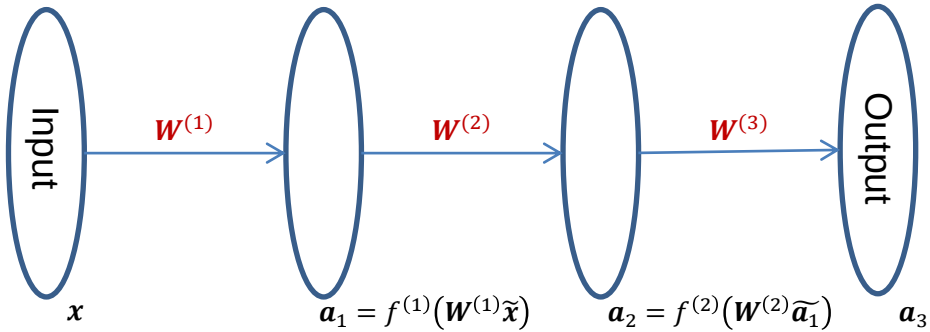
$f^{(2)}\left(\boldsymbol{W}^{(2)}\ \tilde{f}^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right)\right) \in \mathbb{R}^{n_2\times 1}$

$\rightarrow$

$$h(\boldsymbol{x}) = f^{(3)}\left(\boldsymbol{W}^{(3)}\tilde{f}^{(2)}\left(\boldsymbol{W}^{(2)}\ \tilde{f}^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right)\right)\right) \in \mathbb{R}^{M\times 1}$$

where $\tilde{f}^{(2)}\left(\boldsymbol{W}^{(2)}\ \tilde{f}^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right)\right) = \begin{pmatrix} 1 \\ f^{(2)}\left(\boldsymbol{W}^{(2)}\ \tilde{f}^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right)\right) \end{pmatrix} \in \mathbb{R}^{(n_2+1)\times 1}$

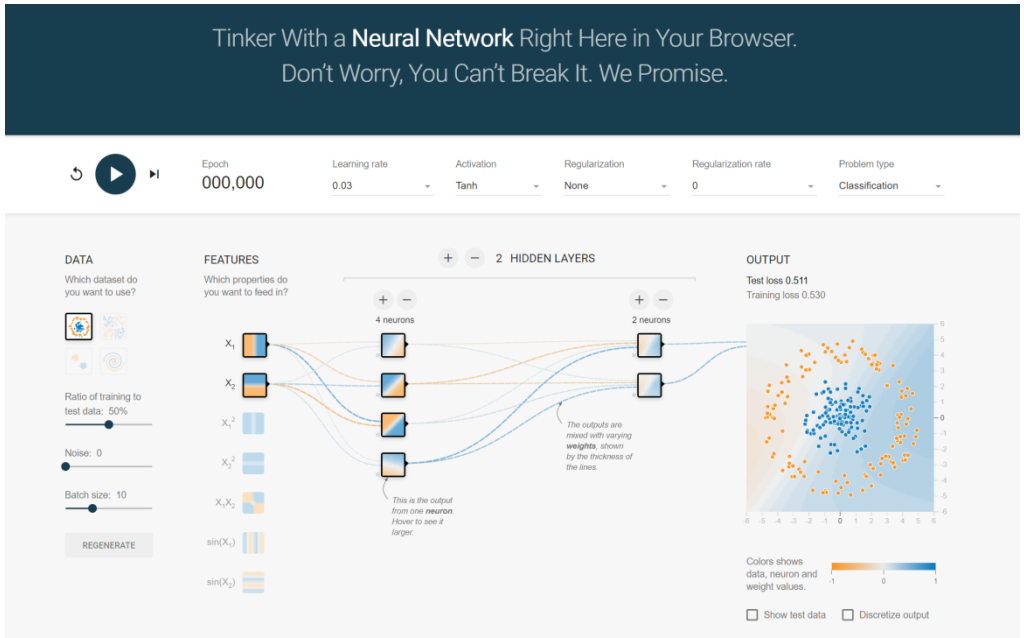
Multilayer perceptron – formula



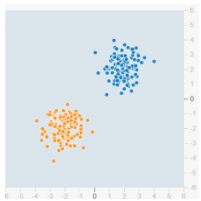
$$h(x) = f^{(3)}\left(W^{(3)}\tilde{f}^{(2)}\left(W^{(2)}\ \tilde{f}^{(1)}(W^{(1)}\tilde{x})\right)\right)$$

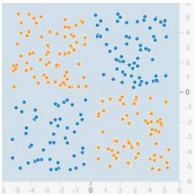
Demo: Tensorflow playground

https://playground.tensorflow.org/



- What is the simplest architecture for these two learning problems?





- Make a DL model with max layers & max nodes (cf. gradient vanishing issue)
  - test sigmoid activation function
  - test ReLu activation function
- Question: why on the *same problem* with *the same* architecture do we get different results?

Back to the loss function J

- The loss function J is used to evaluate the error of the model for a given input.
- It depends on a model (represented by the weights $\boldsymbol{W}$) and a set of input/output samples D_N
  - $J(\boldsymbol{W}, D_N)$
- Many functions can be used,
  - in regression: *mean squared error* (MSE) is the most used
    - $J = \frac{1}{N}\sum_{i=1}^{N}\left(h(\boldsymbol{x}^i) - t^i\right)^2 = \frac{1}{N}\sum_{i=1}^{N} J^i$ where J^i is the error on the sample $(\boldsymbol{x}^i, t^i)$.
  - in classification: the cross-entropy loss is the most used
    - Although technically the MSE can also be used (less strict on errors, poorer performance)

Binary cross-entropy - Commonly used by neural networks for binary-classification problems

Let

- $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ be a dataset with N observations
- $h(x_i) \in [0, 1]$ be the output of the model
- $y_i \in \{0,1\}$ be the true output

Binary cross-entropy:

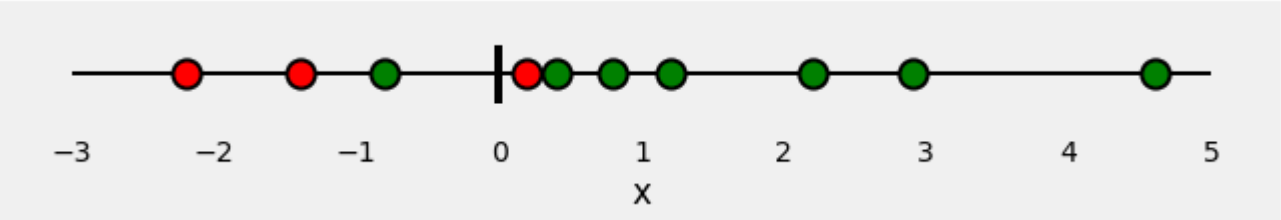
$$CE(h) = -\frac{1}{N}\sum_{i=1}^{N} y_i \,.\log\big(h(x_i)\big) + (1 - y_i)\,.\log(1 - h(x_i))$$

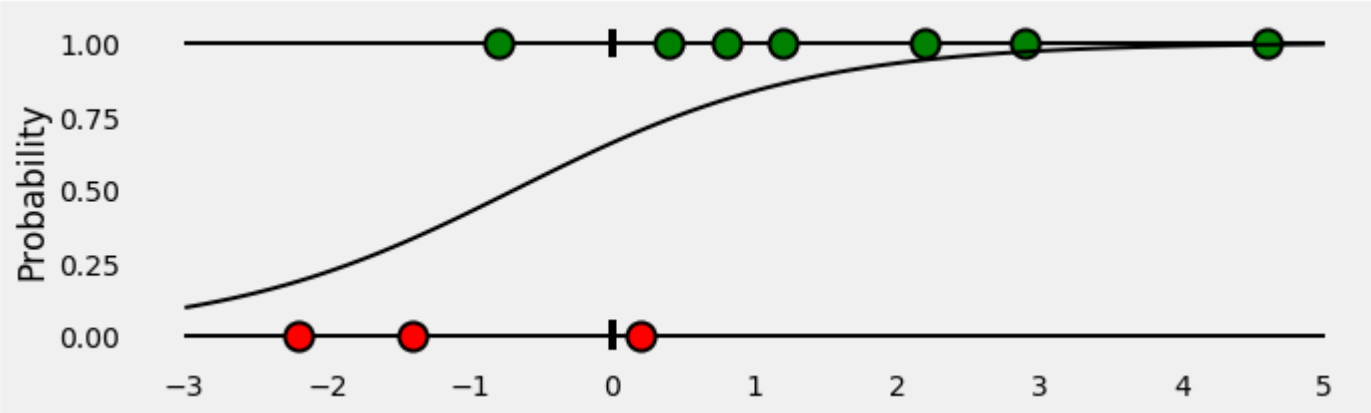
Reading this formula:

- For each "*y=1*", it adds $-1 \times \log\big(h(x_i)\big)$ to the loss.
- For each "*y=0*", it adds $-1 \times \log\big(1 - h(x_i)\big)$ to the loss

Binary cross-entropy — the visual way (1)

Credit : https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a

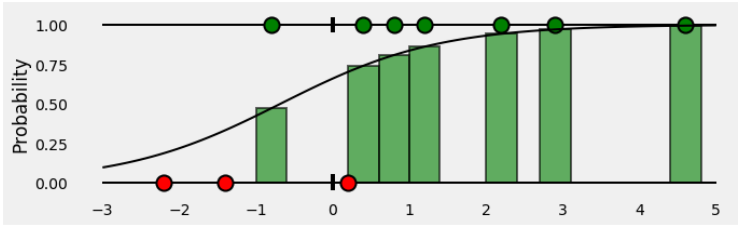


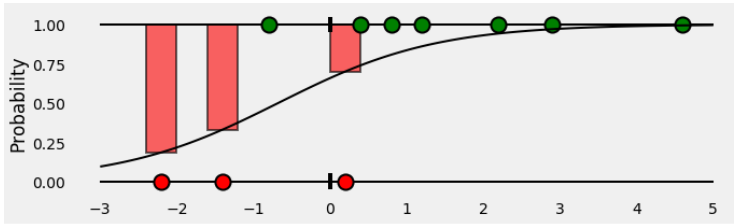


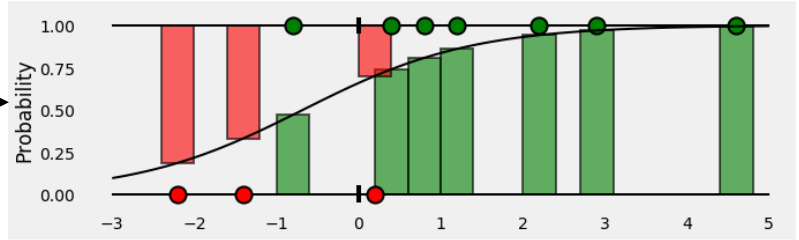
Logistic regression model

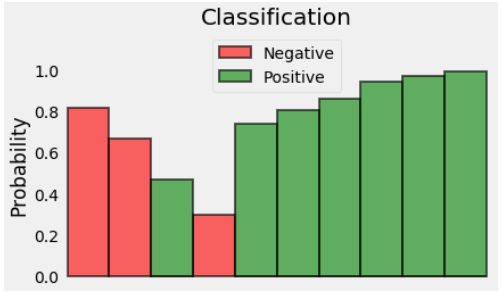
Binary cross-entropy — the visual way (2)

Credit : https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a







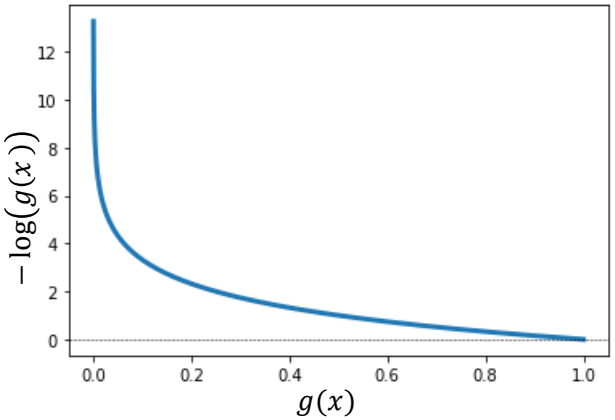


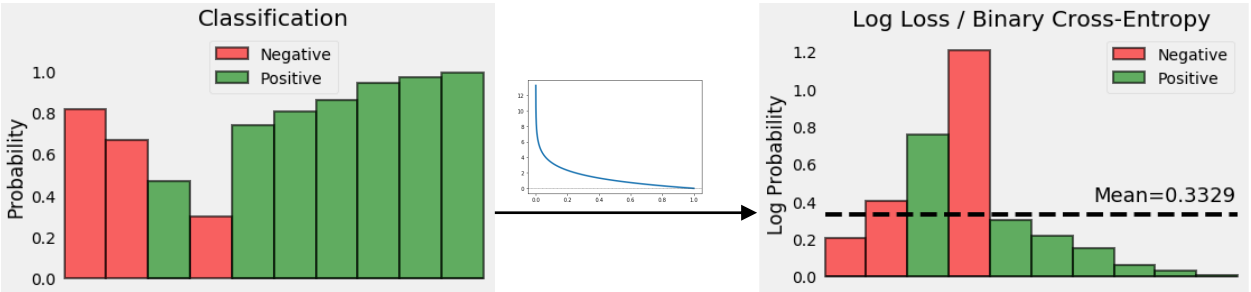
If the **probability** associated with the **true class** is **1.0**, we need its **loss** to be **zero**. Conversely, if that **probability is low**, say **0.01**, we need its **loss** to be **HUGE**!

Binary cross-entropy — the visual way (3)

If the **probability** associated with the **true class** is **1.0**, we need its **loss** to be **zero**.
Conversely, if that **probability is low**, say **0.01**, we need its **loss** to be **HUGE**!

It turns out, taking the **negative log of the probability** suits us well enough for this purpose (*since the log of values between 0.0 and 1.0 is negative, we take the negative log to obtain a positive value for the loss*).





Finally, we compute the **mean of all these losses**.

The **binary cross-entropy** of this example **is 0.3329.**

Binary cross-entropy:

$$CE(h) = -\frac{1}{N}\sum_{i=1}^{N} y_i \,.\log\big(h(x_i)\big) + (1 - y_i)\,.\log(1 - h(x_i))$$

Multi-class cross-entropy loss - Commonly used by neural networks for multi-class problems

$$CE\big(h_y(x_i), p(y|x_i)\big) = -\sum_y p(y|x_i)\ \log\big(h_y(x_i)\big)$$

MODEL

Parameters: θ

	$g_y(x_i)$	LABELS
P(Dog|x_i)	0.80	1.00
P(Cat |x_i)	0.03	0.00
P(Fox|x_i)	0.03	0.00
P(Elephant|x_i)	0.02	0.00
P(Lion|x_i)	0.01	0.00
P(Mouse|x_i)	0.00	0.00
P(Bird|x_i)	0.03	0.00
P(Pokémon|x_i)	0.04	0.00
P(Fish|x_i)	0.01	0.00
P(None|x_i)	0.03	0.00

$h_y(x_i)$	$p(y\|x_i)$
0.7	1
0.2	0
0.1	0
0.9	0

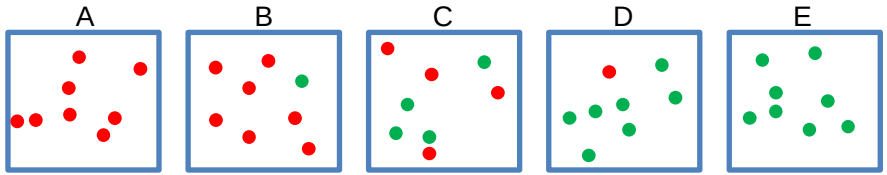
$$CE\big(h_y(x_i), p(y|x_i)\big) = -\log(0.7) = 0.5145$$

Entropy

- Let X be a r.v., the entropy is a measure of the unpredictability of the r.v. X.

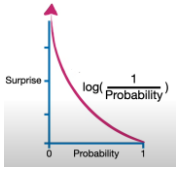
$$H(X) = -\sum_{x \in Supp(X)} P(X = x) \log_2 P(X = x)$$

- It has a lot of applications in data science.
- Like variance, standard deviation & IQR, entropy is a measure of uncertainty.
- Entropy differs since it depends only on the probabilities $P(X = x)$ of the different values and not on the values x themselves.
- If X is a random variable with finite support $\mathfrak{X}$, then X and 100 X have the same entropy, but different variances, standard deviations & IQRs.



- A: there is no surprise in choosing a red. *surprise(red) = 0*
- B: there is no hight surprise in choosing a red but it can be that we select a green
- C: we have the same chance to select a red or a green. *surprise(red) = surprise(green)*
- D: we will be surprised to select a red. *surprise(red)* is hight
- E: it is impossible to select a red. *surprise(red) = undefined*

$$\text{surprise}(x) = \log_2 \frac{1}{P(X = x)}$$



- A: surprise(red)= $\log_2 \frac{1}{1} = 0$
- B: surprise(red)= $\log_2 \frac{1}{7/8} \approx 0.19$
- C: surprise(red)= $\log_2 \frac{1}{4/8} = 1$
- D: surprise(red)= $\log_2 \frac{1}{1/8} = 3$
- E: surprise(red)= $\log_2 \frac{1}{0}$ = undefined

Roughly speaking, entropy is the mean of the "surprise" :

$$H(X) = \sum_{x \in Supp(X)} P(X = x) \log_2 \frac{1}{P(X = x)}$$

Question for you: compute entropy for B.

From entropy to cross entropy

$$H(X) = -\sum_{x \in Supp(X)} P(X = x) \log\ P(X = x)$$

$$CE\left(\boldsymbol{h_y(x_i)}, \boldsymbol{p(y|x_i)}\right) = -\sum_{y} \boldsymbol{p(y|x_i)}\ \log\left(\boldsymbol{h_y(x_i)}\right) = \sum_{y} \boldsymbol{p(y|x_i)}\ \underbrace{\log\left(\frac{1}{\boldsymbol{h_y(x_i)}}\right)}$$

'Surprise' to see y

MLP in TensorFlow

https://github.com/ocaelen/intro_deeplearning/blob/main/many_mlp.ipynb

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

Binary classification

```
model = Sequential([
    Dense(10,input_shape=(4,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.summary()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 10)                50
_________________________________________________________________
dense_1 (Dense)              (None, 6)                 66
_________________________________________________________________
dense_2 (Dense)              (None, 4)                 28
_________________________________________________________________
dense_3 (Dense)              (None, 1)                 5
=================================================================
Total params: 149
Trainable params: 149
Non-trainable params: 0
_________________________________________________________________
```

Multiclass classification

```
model = Sequential([
    Dense(10,input_shape=(4,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(3, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_4 (Dense)              (None, 10)                50
_________________________________________________________________
dense_5 (Dense)              (None, 6)                 66
_________________________________________________________________
dense_6 (Dense)              (None, 4)                 28
_________________________________________________________________
dense_7 (Dense)              (None, 3)                 15
=================================================================
Total params: 159
Trainable params: 159
Non-trainable params: 0
_________________________________________________________________
```

Regression

```
model = Sequential([
    Dense(10,input_shape=(3,), activation='relu'),
    Dense(6, activation='relu'),
    Dense(4, activation='relu'),
    Dense(1)  # Default activation: 'linear'
])
model.summary()
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
hist = model.fit(X, y, epochs=15, verbose=0)
```

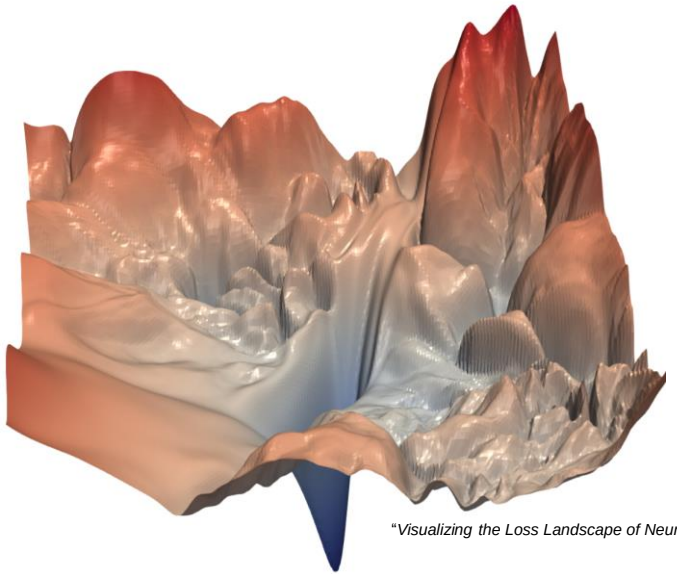
```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_8 (Dense)              (None, 10)                40
_________________________________________________________________
dense_9 (Dense)              (None, 6)                 66
_________________________________________________________________
dense_10 (Dense)             (None, 4)                 28
_________________________________________________________________
dense_11 (Dense)             (None, 1)                 5
=================================================================
Total params: 139
Trainable params: 139
Non-trainable params: 0
_________________________________________________________________
```

Learning in DL context

Learning: Given a training set D_N, search the optimal parameters $\boldsymbol{w}$ minimizing $J(\boldsymbol{w}, D_N)$.

- Learning is an optimization problem.
  - Therefore, many optimization algorithms can be used to find the best $\boldsymbol{w}$.

- For deep learning, the function $J(\boldsymbol{w}, D_N)$ that we have to minimize is too complex to find the global minimum.
  → We need metaheuristic (i.e., find *a good local minimum* rather than the global minimum)

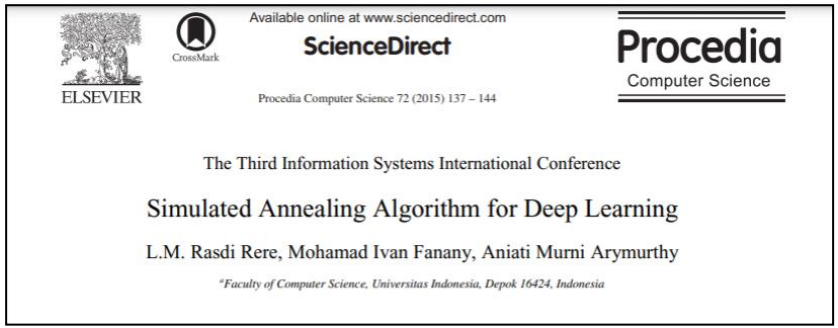


"*Visualizing the Loss Landscape of Neural Nets*" Nov 2017

Learning in DL context

- For deep learning, the most commonly used algorithms are gradient descent algorithms…
  - … and it's what we will see in this lecture!

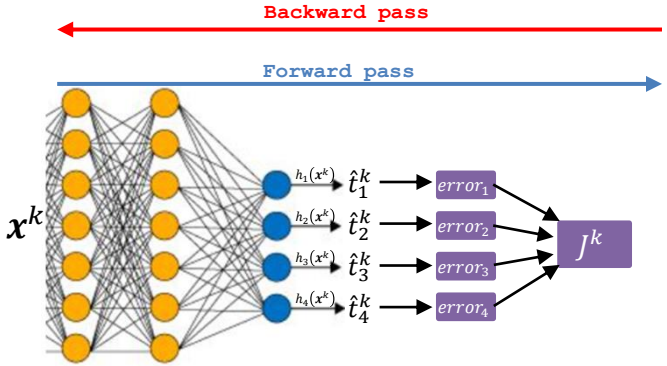
- But other metaheuristic *could* be used (e.g., simulated annealing)
  - but it's still very marginal



Training: Backpropagation

- During the learning process, “coefficients” (synaptic weights & thresholds) are continually updated in an iterative process.
- Gradient descent algorithms are typically used to learn neural network models.
- Backpropagation is a method of calculating the error gradient for each neuron in a neural network, from the last layer to the first.
- *For simplicity*, let us consider 1-sample stochastic gradient descent (i.e., Only one observation used at each step k) and regression.

Training: Backpropagation



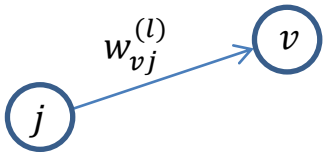
Initializes $\boldsymbol{w}(0)$
Repeat
Select $(\boldsymbol{x}^k, t^k)$ from D_N

- Forward pass
  Compute output $h(\boldsymbol{x}^k)$
- Backward pass
  Compute gradient $\nabla_w J^k$
- Update parameters
  $\boldsymbol{w}(t+1) = \boldsymbol{w}(t) - \alpha \nabla_w J^k$

Let's focus on this step

Training: Backpropagation (compute $\nabla_w J^k$)

- Let J^k be the error of $h(\boldsymbol{x}^k) \in \mathbb{R}^M$ on the training observation $(\boldsymbol{x}^k, \boldsymbol{t}^k)$ where M is the output dimension.
- J^k can be defined as follow: $J^k = \sum_{v=1}^{M} \left(t_v^k - h_v(\boldsymbol{x}^k)\right)^2$.
- Let $w_{vj}^{(l)}$ be the synaptic weight between neuron j and neuron v in layer l.



$$\boldsymbol{W}^{(\boldsymbol{l})} = \begin{pmatrix} w_{1,1}^{(l)} & \cdots & w_{1,(n_{l-1}+1)}^{(l)} \\ \vdots & w_{v,j}^{(l)} & \vdots \\ w_{n_l,1}^{(l)} & \cdots & w_{n_l,(n_{l-1}+1)}^{(l)} \end{pmatrix} \in \mathbb{R}^{n_l \times (n_{l-1}+1)}$$

- $\nabla_{\boldsymbol{w}} \boldsymbol{J}^k = \left(\frac{\partial J^k}{\partial w_{1,1}^{(1)}}, \frac{\partial J^k}{\partial w_{1,2}^{(1)}}, \ldots, \frac{\partial J^k}{\partial w_{n_L,(n_{L-1}+1)}^{(L)}} \right)^T$ → the partial derivative must be computed for all weight $w_{vj}^{(l)}$.
  - the backpropagation does it layer after layer starting at the output.

Reminder – Chain rule

For concreteness, consider the function: $y(x) = e^{\sin(x^2)}$

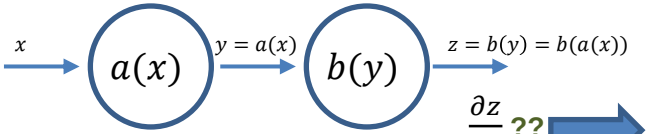
If $\begin{cases} y = f(u) = e^u \\ u = g(v) = \sin(v) \\ v = h(x) = x^2 \end{cases}$, then the chain rule states that $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u}\frac{\partial u}{\partial v}\frac{\partial v}{\partial x}$

As,

- $\frac{\partial y}{\partial u} = f'(u) = e^u = e^{\sin(v)} = e^{\sin(x^2)}$
- $\frac{\partial u}{\partial v} = g'(v) = \cos(v) = \cos(x^2)$
- $\frac{\partial v}{\partial x} = h'(x) = 2x$

$$\longrightarrow \frac{\partial y}{\partial x} = e^{\sin(x^2)}\cos(x^2)\,2x$$

Automatic differentiation & computational graph

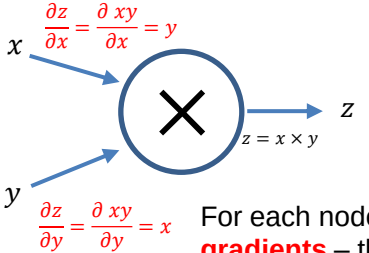


$\frac{\partial z}{\partial x}$??

Chain rule:

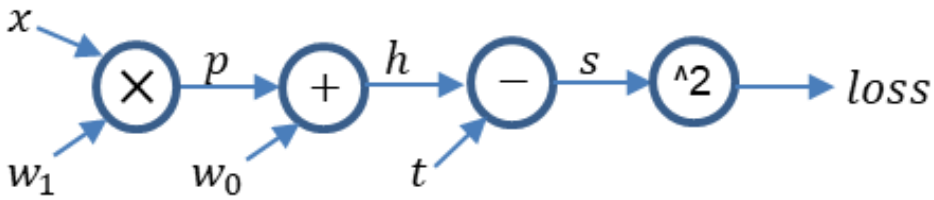
$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

For each operation we perform on tensor, PyTorch and TensorFlow create a graph for us where, at each node, we apply an operation or a function with inputs and obtain outputs.



For each node, we can compute the **local gradients** – that will we used in the chain rule.

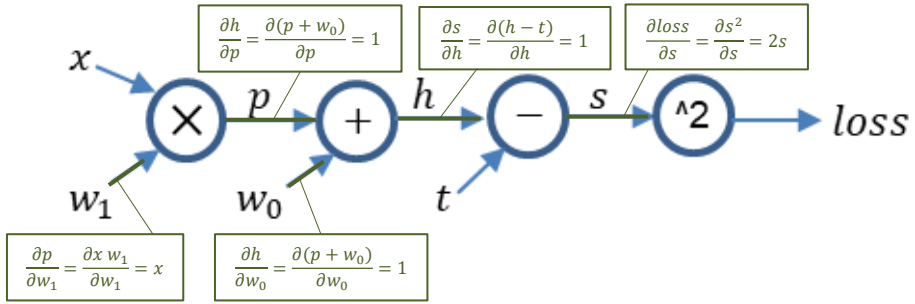
$h(x) = x\, w_1 + w_0$ and $loss = (h(x) - t)^2 = (x\, w_1 + w_0 - t)^2$



$\frac{\partial loss}{\partial w_0}$ = ?? and $\frac{\partial loss}{\partial w_1}$ = ??

Automatic differentiation & computational graph

$h(x) = x\,w_1 + w_0$ and $loss = (h(x) - t)^2 = (x\,w_1 + w_0 - t)^2$



1. Compute all the local gradients in the graph
2. Apply the chain rule
   - $\frac{\partial loss}{\partial w_0} = 2s \times 1 \times 1 = 2s$
   - $\frac{\partial loss}{w_1} = 2s \times 1 \times 1 \times x = 2sx$

Example: $x = 2,\ w_0 = 3,\ w_1 = 4,\ t = 5$

Forward pass → $p = 8, \quad h = 11, \quad s = 6, \quad loss = 36$

→ $\frac{\partial loss}{\partial w_0} = 2 \times 6 = 12, \quad \frac{\partial loss}{\partial w_1} = 2 \times 6 \times 2 = 24$

at $\boldsymbol{w} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$, $\nabla_{\boldsymbol{w}} J = \begin{pmatrix} 12 \\ 24 \end{pmatrix}$

```python
import torch
```

```python
x = torch.tensor(2.0)
t = torch.tensor(5.0)

w0 = torch.tensor(3.0, requires_grad=True)
w1 = torch.tensor(4.0, requires_grad=True)
```

```python
# Forward pass and compute the loss
h = x * w1 + w0
loss = (h - t)**2

print(loss)
```

```
tensor(36., grad_fn=<PowBackward0>)
```

```python
# Backward pass
loss.backward()

print(w0.grad)
print(w1.grad)
```

```
tensor(12.)
tensor(24.)
```

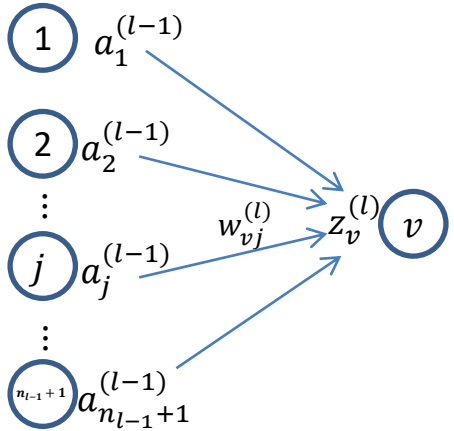
Training: Backpropagation (compute $\frac{\partial J^k}{\partial w_{vj}^{(l)}}$)

Let $z_v^{(l)} = \sum_{j=1}^{n_{l-1}+1} w_{vj}^{(l)} a_j^{(l-1)}$ be the input weighted sum of neuron v in layer l

Thanks to chain rule: $\frac{\partial J^k}{\partial w_{vj}^{(l)}} = \frac{\partial J^k}{\partial z_v^{(l)}} \frac{\partial z_v^{(l)}}{\partial w_{vj}^{(l)}}$

So, it is easy to see that $\frac{\partial z_v^{(l)}}{\partial w_{vj}^{(l)}} = a_j^{(l-1)}$

We still must compute $\frac{\partial J^k}{\partial z_v^{(l)}}$



Two cases :

- l is the output layer (i.e., $l = L$)
- l is not the output layer (i.e., $l < L$)

Training: Backpropagation (compute $\frac{\partial J^k}{\partial z_v^{(l)}}$ when $l = L$)

If l is the output layer then, thanks to chain rule, we have $\frac{\partial J^k}{\partial z_v^{(L)}} = \frac{\partial J^k}{\partial h_v(\boldsymbol{x}^k)} \frac{\partial h_v(\boldsymbol{x}^k)}{\partial z_v^{(L)}}$

As $\frac{\partial J^k}{\partial h_v(\boldsymbol{x}^k)} = \frac{\partial \sum_{v=1}^{M} \left(t_v^k - h_v(\boldsymbol{x}^k)\right)^2}{\partial h_v(\boldsymbol{x}^k)} = -2\left(t_v^k - h_v\left(\boldsymbol{x}^k\right)\right)$

Assuming to be in a regression problem.

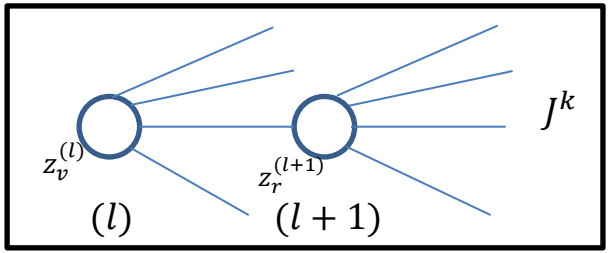
and $\frac{\partial h_v(\boldsymbol{x}^k)}{\partial z_v^{(L)}} = \frac{\partial f^{(L)}(z_v^{(L)})}{\partial z_v^{(L)}} = f'^{(L)}(z_v^{(L)})$

We have: $\boxed{\frac{\partial J^k}{\partial z_v^{(L)}} = -2\left(t_v^k - h_v\left(\boldsymbol{x}^k\right)\right) f'^{(L)}(z_v^{(L)})}$

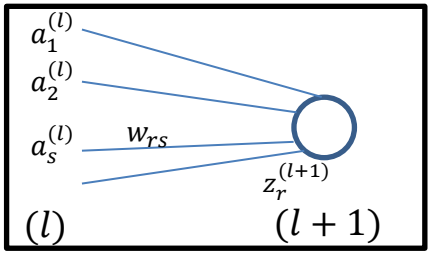
Training: Backpropagation (compute $\frac{\partial J^k}{\partial z_v^{(l)}}$ when $l < L$)

If l isn't the output layer then, thanks to chain rule, we have

$$\frac{\partial J^k}{\partial z_v^{(l)}} = \sum_{r \in (l+1)} \frac{\partial J^k}{\partial z_r^{(l+1)}} \frac{\partial z_r^{(l+1)}}{\partial z_v^{(l)}}$$



- This quantity $\frac{\partial J^k}{\partial z_r^{(l+1)}}$ is already known !!! i.e. '*back propagation of the guardian*'
- $$\frac{\partial z_r^{(l+1)}}{\partial z_v^{(l)}} = \frac{\sum_{s \in (l)} w_{rs}\ a_s^{(l)}}{\partial z_v^{(l)}} = \frac{\sum_{s \in (l)} w_{rs}\ f^{(l)}(z_s^{(l)})}{\partial z_v^{(l)}} = w_{rv}\ f'^{(l)}(z_v^{(l)})$$



We have: $$\frac{\partial J^k}{\partial z_v^{(l)}} = f'^{(l)}\left(z_v^{(l)}\right) \sum_{r \in (l+1)} w_{rv} \frac{\partial J^k}{\partial z_r^{(l+1)}}$$

Training: Backpropagation (in summary)

$$\nabla_{\boldsymbol{w}} \boldsymbol{J}^k = \left(\frac{\partial J^k}{\partial w_{1,1}^{(1)}}, \frac{\partial J^k}{\partial w_{1,2}^{(1)}}, \ldots, \frac{\partial J^k}{\partial w_{vj}^{(l)}}, \ldots, \frac{\partial J^k}{\partial w_{n_L,(n_{L-1}+1)}^{(L)}} \right)^T$$

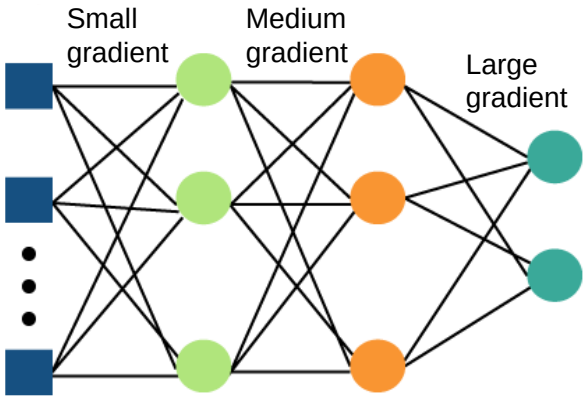
$$\frac{\partial J^k}{\partial w_{vj}^{(l)}} = \frac{\partial J^k}{\partial z_v^{(l)}} a_j^{(l-1)}$$

where $\frac{\partial J^k}{\partial z_v^{(l)}} = -2\left(t_v^k - h_v\left(\boldsymbol{x}^k\right)\right) f'^{(L)}(z_v^{(L)})$ if $\mathrm{l} = L$

$$\frac{\partial J^k}{\partial z_v^{(l)}} = f'^{(l)}\left(z_v^{(l)}\right) \sum_{r \in (l+1)} w_{rv} \frac{\partial J^k}{\partial z_r^{(l+1)}} \quad \text{if } \mathrm{l} < L$$

Gradient vanishing issue

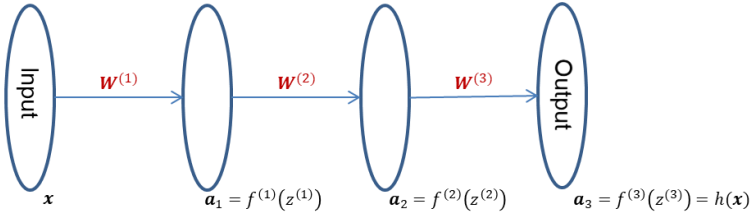
- Deep neural networks with sigmoid or hyperbolic units often suffer from vanishing gradient.



- Further back you go, the gradient tends to be smaller and smaller and smaller and smaller …
- Roughly speaking: If you have a ‘really deep neural network’, the bottom layers do not get trained because the derivatives of those weights are negligible (close to zero).
- In the past, this was a significant problem that prevented researchers from effectively training these multi-layered neural networks.

Gradient vanishing issue with $L = 3$

$$h(\boldsymbol{x}) = f^{(3)}\left(\boldsymbol{W}^{(3)} f^{(2)}\left(\boldsymbol{W}^{(2)}\, f^{(1)}\left(\boldsymbol{W}^{(1)}\tilde{\boldsymbol{x}}\right)\right)\right)$$



$$\frac{\partial J^k}{\partial z_v^{(1)}} = f'^{(1)}\left(z_v^{(1)}\right) \sum_{r\in(l+1)} w_{rv}\, \frac{\partial J^k}{\partial z_r^{(2)}}$$
$$\frac{\partial J^k}{\partial z_v^{(2)}} = f'^{(2)}\left(z_v^{(2)}\right) \sum_{r\in(l+1)} w_{rv}\, \frac{\partial J^k}{\partial z_r^{(3)}}$$
$$\frac{\partial J^k}{\partial z_v^{(3)}} = -2\left(t_v^k - h_v(\boldsymbol{x}^k)\right) f'^{(3)}(z_v^{(3)})$$

$$\frac{\partial J^k}{\partial z_v^{(1)}} = f'^{(1)}\left(z_v^{(1)}\right) \sum_{r\in(l+1)} w_{rv} \left(f'^{(2)}\left(z_v^{(2)}\right) \sum_{r\in(l+1)} w_{rv} \left(2\left(t_v^k - h_v(\boldsymbol{x}^k)\right) f'^{(3)}(z_v^{(3)})\right)\right)$$
$$\frac{\partial J^k}{\partial z_v^{(2)}} = f'^{(2)}\left(z_v^{(2)}\right) \sum_{r\in(l+1)} w_{rv} \left(2\left(t_v^k - h_v(\boldsymbol{x}^k)\right) f'^{(3)}(z_v^{(3)})\right)$$
$$\frac{\partial J^k}{\partial z_v^{(3)}} = -2\left(t_v^k - h_v(\boldsymbol{x}^k)\right) f'^{(3)}(z_v^{(3)})$$

To compute the gradient at the input layer, the derivative of the activation function is called L times. Roughly speaking we have:

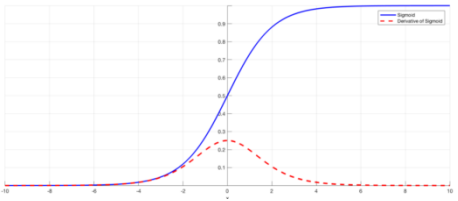
$$\frac{\partial J(\boldsymbol{W}, \boldsymbol{x}, \boldsymbol{y})}{\partial w_{vj}^{(1)}} \propto (\ldots) f'^{(L)}\left(z^{(L)}\right) \times (\ldots) f'^{(L-1)}\left(z^{(L-1)}\right) \times \cdots \times (\ldots) f'^{(2)}\left(z^{(2)}\right) \times (\ldots) f'^{(1)}\left(z^{(1)}\right)$$

Gradient vanishing issue

- To compute the gradient at the input layer, the derivative of the activation function is called L times. Roughly speaking we have:

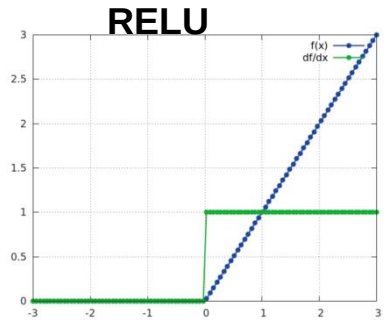
$$\frac{\partial J(\boldsymbol{W},\boldsymbol{x},\boldsymbol{y})}{\partial w_{vj}^{(1)}} \propto (\dots) f'^{(L)}\left(z^{(L)}\right) \times (\dots) f'^{(L-1)}\left(z^{(L-1)}\right) \times \cdots \times (\dots) f'^{(2)}\left(z^{(2)}\right) \times (\dots) f'^{(1)}\left(z^{(1)}\right)$$

- If the $f'^{()}\left(z_v^{()}\right)$ are smaller than one then $\frac{\partial J(\boldsymbol{W},\boldsymbol{x},\boldsymbol{y})}{\partial w_{vj}^{(1)}}$ converge exponentially rapidly to 0 when L increase.
  - example if a=0.2 then $a^{10} \approx 10^{-7}$



- The maximum of the derivative of the sigmoidal function in 0.25
  - Gradient vanishing issue

- Several popular solutions to avoid vanishing issue :
  - Pre-training
  - **Rectified linear units (RELU)**
  - Skip connections
  - Batch normalization

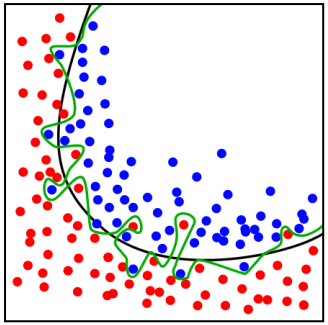


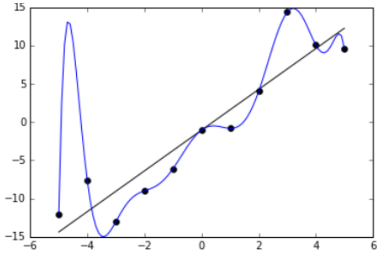
- but the RELU derivative is 0 if input negative - ‘dead neuron’ issue
  - cf Demo: Tensorflow playground

Overfitting

Overfitting is "the production of an analysis that corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably“

(Definition of "overfitting" at OxfordDictionaries.com)





While the green (classification) & blue (regression) lines best follows the training data, it is too dependent on that data and it is likely to have a higher error rate on new unseen data, compared to the black line.

- Roughly speaking, overfitting occurs when the model class (i.e., the architecture) is too complex for the learning problem.
  → the algorithm starts to learn the noise
- Overfitting can be measure during learning via a validation set (no used during gradient descent).
- To avoid it:
  - Regularization, manly L1 and L2
  - Cross validation, to search optimal architecture
    - difficult in deeplearning
  - Early stopping
  - Dropout nodes in the deeplearning architecture
  - …

How to deal with overfitting ?

Options:

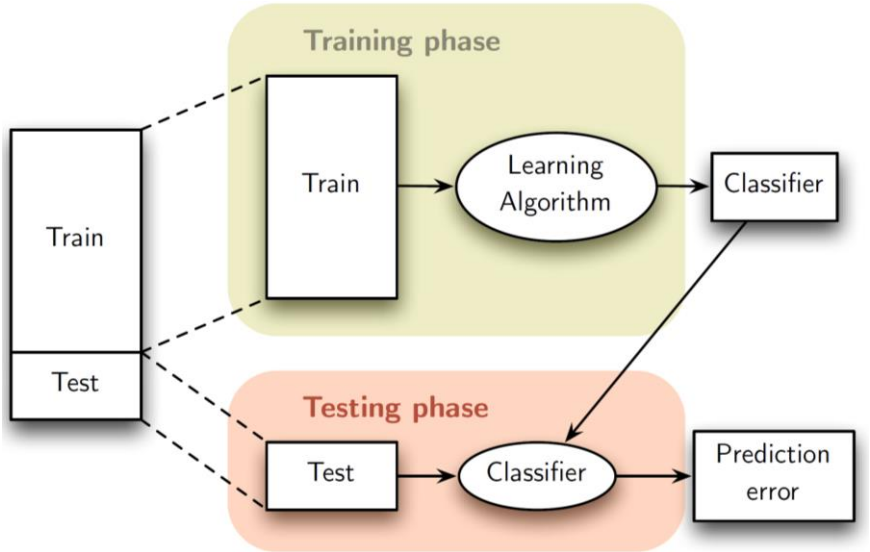
- Reducing the number of parameters (~complexity)
  - Model selection (Validation set, Cross validation, …)
    - Traditional method but difficult with DL

- Keep all parameters with …
  - Early stopping
  - Dropout
  - Regularization
  - …

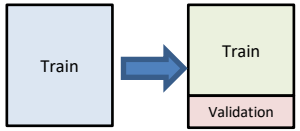
Training, validation testing [reminder]

Given:
- a machine learner (e.g., SVM, DNN, KNN, …)
- a set of hyperparameters. E.g.,
  - $\Lambda_1 = \{L = 2, n_1 = 4, n_2 = 2\}$
  - $\Lambda_2 = \{L = 1\ n_1 = 2\ \}$
  - …

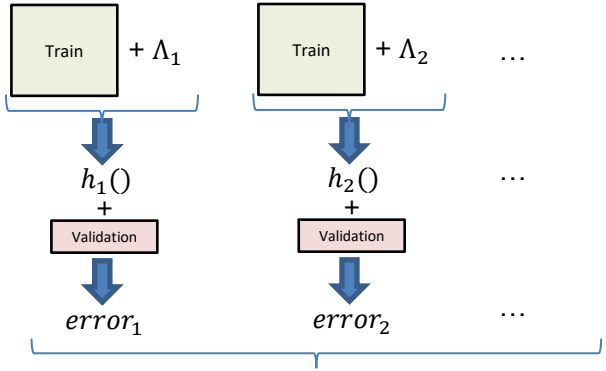
How to find the best predictive model $h(x)$?



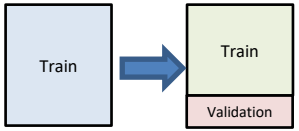
- The testing set is used at end to have a fair estimate of the predictive model accuracy if production context.
- It is absolutely forbidden to touch the test set during learning (i.e., optimization)! (why?)
- During training phase, the same train set must be used for both:
  - Selection of best hyperparameters (i.e. , Λ)
  - Selection of best paraments (i.e., $\boldsymbol{W}$)
- Need to split training set in training and validation sets



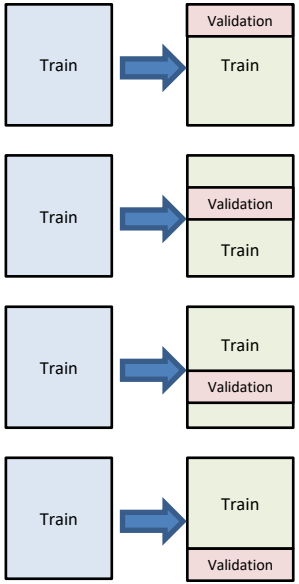
- The new train and validation sets can now be used to identify the best hyperparameters.



Cross validation



Data is subdivided into k cross-validation folds (typically k = 5 or k = 10, or leave-one-out cross validation)



Model is trained using k-1 folds and tested on the remaining fold

```
>>> from sklearn import datasets, linear_model
>>> from sklearn.model_selection import cross_val_score
>>> diabetes = datasets.load_diabetes()
>>> X = diabetes.data[:150]
>>> y = diabetes.target[:150]
>>> lasso = linear_model.Lasso()
>>> print(cross_val_score(lasso, X, y, cv=3))
[0.33150734 0.08022311 0.03531764]
```

https://scikit-learn.org/

```
>>> from sklearn import svm, datasets
>>> from sklearn.model_selection import GridSearchCV
>>> iris = datasets.load_iris()
>>> parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
>>> svc = svm.SVC()
>>> clf = GridSearchCV(svc, parameters)
>>> clf.fit(iris.data, iris.target)
GridSearchCV(estimator=SVC(),
             param_grid={'C': [1, 10], 'kernel': ('linear', 'rbf')})
>>> sorted(clf.cv_results_.keys())
['mean_fit_time', 'mean_score_time', 'mean_test_score',...
 'param_C', 'param_kernel', 'params',...
 'rank_test_score', 'split0_test_score',...
 'split2_test_score', ...
 'std_fit_time', 'std_score_time', 'std_test_score']
```

https://scikit-learn.org/

Process is repeated k times, prediction errors are averaged

After identifying the best hyperparameters, we can use all the training samples to build the final predictive model.

class sklearn.model_selection.**GridSearchCV**(*estimator, param_grid, *, scoring=None, n_jobs=None, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False*)

Train + best hyperparameters → Final predictive model

It is only now, when the whole optimization process is finished, that we can evaluate our *final predictive model* with the test set.

Validation

What is the purpose of having a validation set for a model?

1. To test the model by seeing how it performs on some "hard cases" in the dataset.
2. To check the performance of the model and remove the need for a test set.
3. To provide some extra data that can be used to train the model.
4. To evaluate the generalisation performance of the model, check for overfitting, and allow for hyperparameter tuning.

Cross validation & deep learning

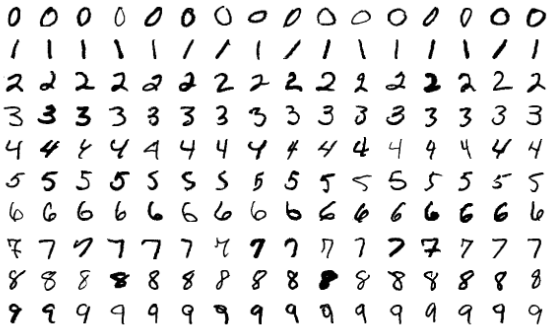
Question for you: In practice, it is difficult to use the CV with DL… why?

https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_overfitting.ipynb

Overfitting – validation set

In [2]:
```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Scale images variables in [0, 1]
x_train = x_train.astype("float32") / 255
x_test = x_test.astype("float32") / 255

# shape (60000, 28, 28) -> (60000, 28, 28, 1)
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

# shape (60000,) -> (60000, 1)
y_train = y_train.reshape(-1,1)
y_test = y_test.reshape(-1,1)

# output to one hot encoding
enc = OneHotEncoder(sparse=False)
y_train = enc.fit_transform(y_train)
y_test = enc.transform(y_test)
```

MNIST

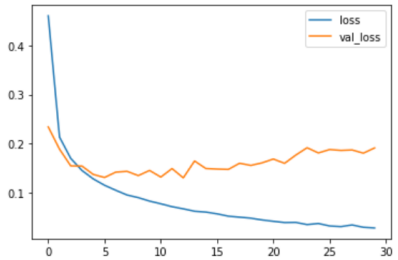


In [3]:
```
model = Sequential([
    Flatten(input_shape=(28,28,1)),
    Dense(30, activation='relu'),
    Dense(20, activation='relu'),
    Dense(15, activation='relu'),
    Dense(10, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2)
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 30)                23550
_________________________________________________________________
dense_1 (Dense)              (None, 20)                620
_________________________________________________________________
dense_2 (Dense)              (None, 15)                315
_________________________________________________________________
dense_3 (Dense)              (None, 10)                160
=================================================================
Total params: 24,645
Trainable params: 24,645
Non-trainable params: 0
```

In [4]:
```
pd.DataFrame(hist.history).plot()
```

Out[4]: <AxesSubplot:>



In [5]:
```
model.evaluate(x_test, y_test)
```

```
313/313 [==============================] - 1s 5ms/step - loss: 0.1920
```

Out[5]: 0.1919853687286377

https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_early_stopping.ipynb

Overfitting – Early stopping (i)

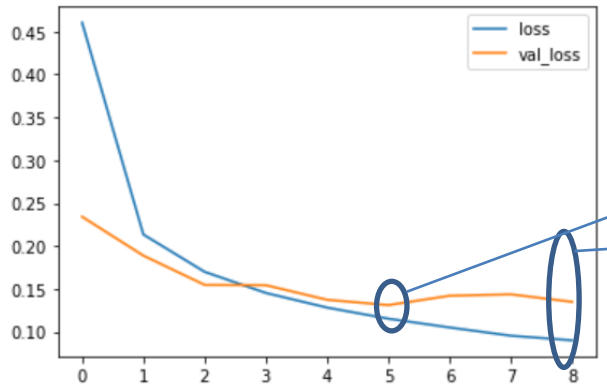
In [3]:
```
early_stopping = EarlyStopping(monitor='val_loss',
                               patience=3,
                               verbose=1)
checkpoint = ModelCheckpoint(filepath = 'checkpoint_best',
                             monitor='val_loss',
                             save_best_only=True,
                             save_weights_only=True,
                             verbose=1)
```

In [4]:
```
model = Sequential([
    Flatten(input_shape=(28,28,1)),
    Dense(30, activation='relu'),
    Dense(20, activation='relu'),
    Dense(15, activation='relu'),
    Dense(10, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2,
                 callbacks=[early_stopping, checkpoint])
```

```
Epoch 1/30
1499/1500 [============================>.] - ETA: 0s - loss: 0.4605
Epoch 00001: val_loss improved from inf to 0.23411, saving model to checkpoint_best
1500/1500 [==============================] - 17s 11ms/step - loss: 0.4604 - val_loss: 0.2341
Epoch 2/30
1499/1500 [============================>.] - ETA: 0s - loss: 0.2133
Epoch 00002: val_loss improved from 0.23411 to 0.18881, saving model to checkpoint_best
1500/1500 [==============================] - 17s 11ms/step - loss: 0.2134 - val_loss: 0.1888
Epoch 3/30
1497/1500 [============================>.] - ETA: 0s - loss: 0.1701
Epoch 00003: val_loss improved from 0.18881 to 0.15467, saving model to checkpoint_best
1500/1500 [==============================] - 16s 11ms/step - loss: 0.1699 - val_loss: 0.1547
Epoch 4/30
1497/1500 [============================>.] - ETA: 0s - loss: 0.1449
Epoch 00004: val_loss improved from 0.15467 to 0.15432, saving model to checkpoint_best
1500/1500 [==============================] - 19s 13ms/step - loss: 0.1453 - val_loss: 0.1543
Epoch 5/30
1499/1500 [============================>.] - ETA: 0s - loss: 0.1281
Epoch 00005: val_loss improved from 0.15432 to 0.13729, saving model to checkpoint_best
1500/1500 [==============================] - 19s 12ms/step - loss: 0.1282 - val_loss: 0.1373
Epoch 6/30
1500/1500 [==============================] - ETA: 0s - loss: 0.1153
Epoch 00006: val_loss improved from 0.13729 to 0.13111, saving model to checkpoint_best
1500/1500 [==============================] - 21s 14ms/step - loss: 0.1153 - val_loss: 0.1311
Epoch 7/30
1493/1500 [============================>.] - ETA: 0s - loss: 0.1048
Epoch 00007: val_loss did not improve from 0.13111
1500/1500 [==============================] - 15s 10ms/step - loss: 0.1050 - val_loss: 0.1420
Epoch 8/30
1496/1500 [============================>.] - ETA: 0s - loss: 0.0954
Epoch 00008: val_loss did not improve from 0.13111
1500/1500 [==============================] - 18s 12ms/step - loss: 0.0954 - val_loss: 0.1437
Epoch 9/30
1497/1500 [============================>.] - ETA: 0s - loss: 0.0899
Epoch 00009: val_loss did not improve from 0.13111
1500/1500 [==============================] - 17s 11ms/step - loss: 0.0900 - val_loss: 0.1349
Epoch 00009: early stopping
```

Overfitting – Early stopping (ii)

```
In [5]: pd.DataFrame(hist.history).plot()
Out[5]: <AxesSubplot:>
```



Save the model minimizing val_loss

```
checkpoint = ModelCheckpoint(filepath = 'checkpoint_best',
                             monitor='val_loss',
                             save_best_only=True,
                             save_weights_only=True,
                             verbose=1)
```

Stop the learning process after 3 consecutive increases in val_loss

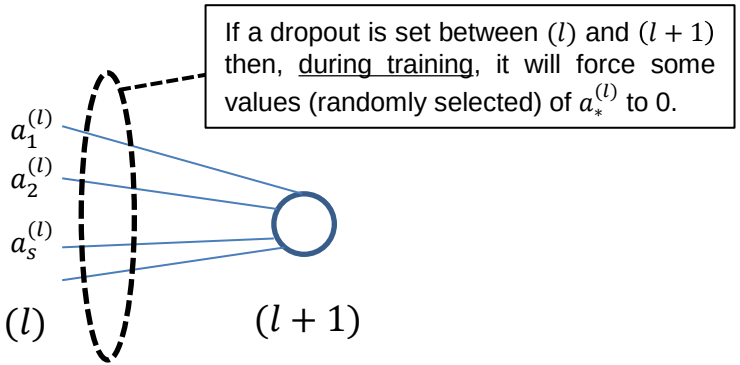
```
early_stopping = EarlyStopping(monitor='val_loss',
                               patience=3,
                               verbose=1)
```

```
In [6]: model.load_weights('checkpoint_best')
        model.evaluate(x_test, y_test)

        313/313 [==============================] - 2s 6ms/step - loss: 0.1296
Out[6]: 0.12957791984081268
```

Reload the best model minimizing val_loss

Overfitting – Dropout (i)

- At each step during the training time, the **dropout layer** randomly sets the input nodes ("a") to 0 with a given frequency.



- The input nodes not set to 0 are scaled up by 1/(1 - rate) such that the sum over all inputs is unchanged.
- The Dropout layer is inactive during predictions.
- It helps to generalize by preventing a node from becoming too important in the prediction.
- It can also be interpreted as a way to learn in parallel a set of neural networks with different architectures.
  - a kind of bagging technique like random forest

Overfitting – Dropout (ii)

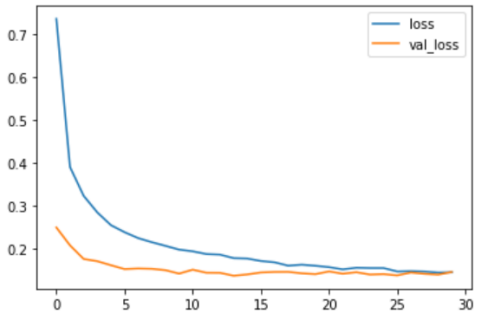
https://github.com/ocaelen/intro_deeplearning/blob/main/tf_simple_dropout.ipynb

In [3]:
```
model = Sequential([
    Flatten(input_shape=(28,28,1)),
    Dense(30, activation='relu'),
    Dropout(0.1),
    Dense(20, activation='relu'),
    Dropout(0.1),
    Dense(15, activation='relu'),
    Dropout(0.1),
    Dense(10, activation='softmax')
])
model.summary()
model.compile(optimizer='adam', loss='categorical_crossentropy')
hist = model.fit(x_train, y_train, epochs=30, verbose=1, validation_split=0.2)
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 30)                23550
_________________________________________________________________
dropout (Dropout)            (None, 30)                0
_________________________________________________________________
dense_1 (Dense)              (None, 20)                620
_________________________________________________________________
dropout_1 (Dropout)          (None, 20)                0
_________________________________________________________________
dense_2 (Dense)              (None, 15)                315
_________________________________________________________________
dropout_2 (Dropout)          (None, 15)                0
_________________________________________________________________
dense_3 (Dense)              (None, 10)                160
=================================================================
Total params: 24,645
Trainable params: 24,645
Non-trainable params: 0
```

In [4]:
```
pd.DataFrame(hist.history).plot()
```

Out[4]: <AxesSubplot:>



In [5]:
```
model.evaluate(x_test, y_test)
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.1505
```

Out[5]: 0.15052251517772675

Regularization

We keep all parameters but reduce the magnitude of the parameters $w_{vj}^{(l)}$.

A regularization term $R(h)$ is added to the loss function

$$J_R(\boldsymbol{w}, D_N) = \left(\frac{1}{N}\sum_{i=1}^{N} J^i\right) + \lambda R(h)$$

where J^i is the underlying loss that describes the cost of predicting $h(\boldsymbol{x}^i)$ when the true output is t^i and λ is a parameter which controls the importance of the regularization term $R(h)$.

Given the training set D_N, during learning we search the optimal parameters $\boldsymbol{w}$ minimizing this new regularized loss function.

$$\widehat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}} \left(\frac{1}{N}\sum_{i=1}^{N} J^i\right) + \lambda R(h)$$

L1 & L2 Regularization

The key difference between L1 & L2 is the penalty term.

$$J_{l1}(\boldsymbol{w}, D_N) = \left(\frac{1}{N}\sum_{i=1}^{N} J^i\right) + \lambda \sum_j \|w_j\| \qquad J_{l2}(\boldsymbol{w}, D_N) = \left(\frac{1}{N}\sum_{i=1}^{N} J^i\right) + \lambda \sum_j (w_j)^2$$

- If $\lambda = 0$ then we get back the original loss function (overfitting if lot of parameters).
- However, if λ is very large then it will add too much weight and it will lead to underfitting.

Back to Single-layer Neural Networks (i.e., Linear regression & logistic regression)

- If L1 regularization, then “Lasso Regression”
- If L2 regularization, then “Ridge Regression”

It is essential to normalize or standardize the input variables!

$$\boldsymbol{X} = \begin{pmatrix} x_1^1 & \cdots & x_n^1 \\ \vdots & \ddots & \vdots \\ x_1^N & \cdots & x_n^N \end{pmatrix} = (\boldsymbol{x}_1 \quad \ldots \quad \boldsymbol{x}_n) \in \mathbb{R}^{N \times n}$$

- Each column of the training dataset can have different quantities with very different ranges of values.
  - size in meters : 1.5 → 2.2
  - weight in kg: 50 → 180
  - incomes in €: 700 → 4000
- the objective is to bring all variables to the same scale.
- Normalize:

$$\tilde{x}_{\mathrm{j}}^{\mathrm{i}} = \frac{x_j^i - \min(\boldsymbol{x}_j)}{\max(\boldsymbol{x}_j) - \min(\boldsymbol{x}_j)}$$

- Standardize:

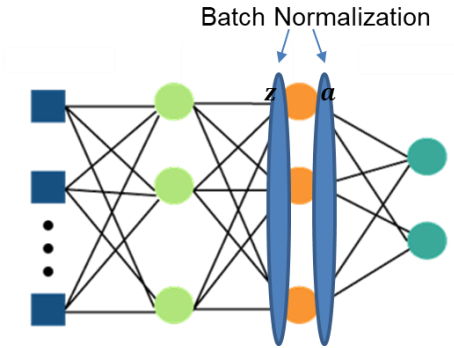
$$\tilde{x}_{\mathrm{j}}^{\mathrm{i}} = \frac{x_j^i - \mathrm{avg}(\boldsymbol{x}_j)}{\mathrm{sd}(\boldsymbol{x}_j)}$$

- It helps improve model performance, training stability, and convergence speed.
- By scaling inputs to a consistent range gradient descent optimization becomes more efficient, preventing large weights.
- This uniformity in scale ensures that each feature contributes equally to the learning process, reducing the risk of certain features dominating others.

Batch Normalization

"*Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*" (2015)

- The learning performance is improved when the input variables are normalized.
  - Why not apply the same normalization operation to the hidden layers?
- The purpose of this idea is to normalize earth the $a = f(\boldsymbol{w}^T\tilde{\boldsymbol{x}})$ **or** the $z = \boldsymbol{w}^T\tilde{\boldsymbol{x}}$.
  - In the original paper, it is on the z.
- Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information.



Batch Normalization

Let's assume that:
- we normalize the z
- $|\mathcal{B}|$ is the batch size

The Batch Normalization is defined as follow:

$$z_{norm} = \frac{z - \hat{\mu}_z}{\sqrt{\hat{\sigma}^2 + \epsilon}}; \quad \text{where } \hat{\mu}_z = \frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|} z_i \quad and \quad \hat{\sigma}^2 = \frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}(z_i - \hat{\mu}_z)^2$$

and

$$\tilde{z}_{norm} = \gamma \,.\, z_{norm} + \beta$$

This is done for each neuron in the layer.
The ϵ is added in the denominator for numerical stability and is an arbitrarily small constant.
γ and β are learnable parameters.

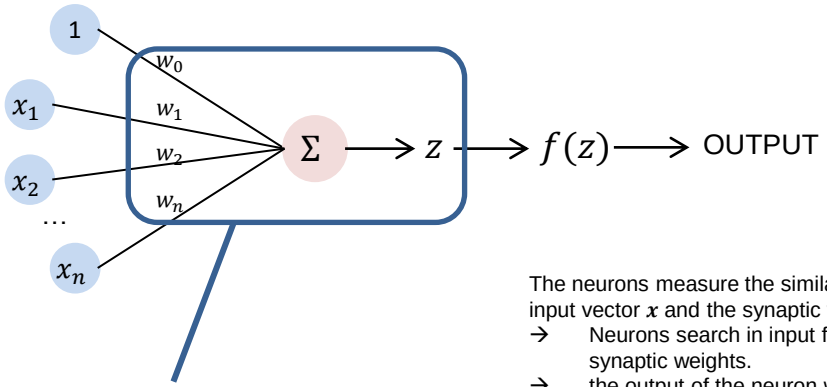
Note that because we subtract the mean $\hat{\mu}_z$, we don't really need extra bias parameter w_0 in the neurone because it is loss.
→ $z - \hat{\mu}_z$ doesn't depend on the bias, and so z_{norm} doesn't depend on the bias. The bias becomes irrelevant.
→ β takes the role of the bias

Batch Normalization – during training & inference

- **During training**:
  - Batch normalization helps to stabilize and speed up the learning process by normalizing the inputs of layers.
  - After normalization, a learnable scale γ and shift parameter β is applied to maintain the representation power of the model.
  - The mean and variance of each batch are also used to compute a running average, which is an estimate of the overall mean and variance of the dataset.

- **During inference (or the test phase)**:
  - The batch normalization layer behaves differently.
  - Since we usually evaluate or predict one sample at a time, it's not appropriate to compute the mean and variance from the current 'test batch'.
  - Instead, we use the running average of mean and variance calculated during the training phase to normalize the inputs.
  - This running average serves as an estimate of the population means and variance and ensures that the network's behaviour remains consistent between training and inference.

The inner product as similarity measure

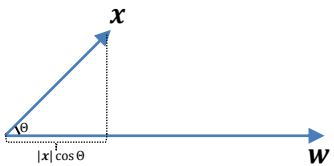


The neurons measure the similarity between the neuron's input vector $\boldsymbol{x}$ and the synaptic weights $\boldsymbol{w}$.

→ Neurons search in input for patterns defined by synaptic weights.

→ the output of the neuron will be large if the input is like the pattern defined by the weights.

$$\boldsymbol{w}^T \tilde{\boldsymbol{x}}^i = |\boldsymbol{w}|\,|\boldsymbol{x}|\,\boldsymbol{cos}(\boldsymbol{\theta})$$



Inner products are often used as a **measure of similarity between two vectors** because they can provide a concise and computationally efficient way to **compare the directions of the vectors**.

When two vectors are combined through an inner product, the result is a scalar value that reflects the extent to which the vectors are pointing in the same direction.

If the two vectors are pointing in the same direction, the angle between them will be 0 degrees, and the cos(0)=1. This means that the inner product of the two vectors will be equal to the product of their magnitudes, **which is at its maximum value**.

On the other hand, **if the two vectors are orthogonal** (i.e., perpendicular) to each other, the angle between them is 90 degrees, and the cosine of 90 is 0. This means that **the inner product of the two vectors will be 0**, which indicates that they are not similar at all.

In general, the larger the inner product of two vectors, the more similar they are in terms of direction.